Lecture Notes LP: Lyon 2026

Sophie Huiberts

2026-11-06

We will cover linear programming from a theoretical perspective, with a focus on the simplex method. Topics will include:

- What are linear programs, what are they good for, what are they bad for
- Fundamental concepts such as duality
- Convex polyhedra, their combinatorics and geometry
- The simplex method (theoretically)
- Why the simplex method is slow
- Why the simplex method is fast
- If we have time: the simplex method (practically)

Despite the linear programming and the simplex method predating modern computers, this is still an active area of research. There are many algorithms that work 'better' than theory can currently explain. The simplex method is one of those, although it is perhaps the best understood among the poorly understood algorithms.

0.1. Promises about notation

Sets of integers are denoted $[k] = \{1, .., k\}$.

We will often need to index into matrices or vectors. This is done using subscripts. For a vector $b \in \mathbb{R}^n$ and index $i \in [n]$, the i'th entry of b is denoted as $b_i \in \mathbb{R}$. Similarly, if $S \subset [n]$ then we denote the restricted vector by $b_S \in \mathbb{R}^S$.

When we index into a matrix, we only ever select rows (selecting columns would be ridiculous). That is, when $A \in \mathbb{R}^{n \times d}$ and $i \in [n]$ then $A_i \in \mathbb{R}^d$ is the i'th row of A. Similarly, if $S \subset [n]$ then we denote the submatrix by $A_S \in \mathbb{R}^{S \times [d]}$.

For those who have not seen this notation before. When we have sets K, L then K^L denotes the set of functions from L to K. Hence if k is a natrual number and $h \in \mathbb{R}^{[k]}$, then k is a function that assigns to each index $i \in [k]$ a real number in $\mathbb{R}$. In other words, h is a vector. When we write a real vector space as $\mathbb{R}^k$, then that is effectively shorthand for $\mathbb{R}^{[k]}$. Similarly, $\mathbb{R}^{n \times d}$ we can think of as shorthand for $\mathbb{R}^{[n] \times [d]}$.

There may be times when you are tempted to believe that $n \geq d$ implies that $[d] \subseteq [n]$. Resist this temptation; the consequent is syntactically correct but semantically worse than useless.

1. Linear Programming

When we talk about a linear program, we are talking about any problem that can be written in the following form:

$$\begin{aligned} &\text{maximize } c^T x \\ &\text{subject to } Ax \leq b. \end{aligned} \tag{1}$$

The program is specificied by the *constraint matrix* $A \in \mathbb{R}^{n \times d}$, the *right-hand side vector* $b \in \mathbb{R}^n$ and the *objective vector* $c \in \mathbb{R}^d$. A *feasible solution* to the program is any vector $x \in \mathbb{R}^d$ that satisfies the system of coordinate-wise linear inequalities $Ax \leq b$. That is, if we denote the rows of A as row vectors $A_1, A_2, ..., A_n$, then x must satisfy $A_i x \leq b_i$ for all $i = 1, ..., n$. No feasible solution need exist.

An *optimal feasible solution* is any feasible solution $x \in \mathbb{R}^d$ such that for any other feasible solution $x' \in \mathbb{R}^d$ we have $c^T x \geq c^T x'$. That is, among all feasible solutions, x should have an *objective value* that is the highest possible. No optimal feasible solution need exist.

The input data of the LP is A, b, c, and the computational task described by this data is to find an optimal feasible solution x or prove that none exists.

1.1. Graphical example

We drew an LP in 2 variables on the board. The feasible set is the intersection of a number of (affine) half-spaces. The result is a convex polyhedron (in 2 variables, a polygon). An optimal feasible solution is a point that is the furthest possible in the direction of the objective vector. In 3 variables we drew a three-dimensional cube as a feasible region. In either case, we agree that (under all reasonable circumstances) if there is an optimal feasible solution then there is one that is also a vertex of the feasible polyhedron.

1.2. LP on the computer

Most LPs are there to be solved. A user writes (a program to construct) the input data A, b, c in the computer, and a dedicated software library is called to return an optimal feasible solution x. In this role, LP is an important piece of infrastructure in industrial societies. It is used to design and operate national power systems, in forest management, in petroleum refinery, and when cutting paper, metal or glass. In biology, you can find LP in flux balance analysis. In data science, LP underlies least absolute error regression. If you manufacture a product, you may use LP when moving products from factories to warehouses. And on your own computer and phone, LP is used when drawing windows and when stabilizing video. LP is also an essential part of the dominant paradigms for solving mixed integer (non-)linear mixed integer programming problems, a technology with even broader applications.

In all these applications, the simplex method is an essential part of all available software packages. Sophisticated software may include additional algorithms for LP on top of a simplex method, such as an interior-point method or a first-order method.

1.3. LP as mathematical objects

LPs can be solved, but they are also mathematical objects with rich structure that can be studied in their own right. In this course we will see some of this structure. This will include how to certify whether a feasible solution is optimal and how to certify whether or not any feasible solution exists. The structure of LP solutions (strong duality, complementary slackness) is a useful feature in numerous mathematical proofs.

The various algorithms for LP motivate mathematical questions as well.

- For the simplex method, we will see in this course a number of questions in convex geometry and in polyhedral combinatorics.
- In algorithm theory, it is known that LPs can be solved in weakly polynomial time. This was first proven for the ellipsoid method, a theoretical algorithm that works very poorly in practice. It is unknown whether LPs can be solved in strongly polynomial time.
- The other well-known class of weakly polynomial time algorithms are the interior-point methods. These can be competitive in practice with simplex methods. IPMs have a similarly rich theory as simplex methods do.

1.4. We Don't Talk About Standard Form

Other textbooks and resources may consider other forms of LP. This includes what is called "standard form"

$$\begin{aligned} &\text{minimize } c^T x \\ &\text{subject to } Ax = b \\ &\qquad\qquad x \geq 0, \end{aligned} \tag{2}$$

and the similarly non-descriptively named "canonical form"

$$\begin{aligned} &\text{maximize } c^T x \\ &\text{subject to } Ax \leq b \\ &\qquad\qquad x \geq 0. \end{aligned} \tag{3}$$

Please convince yourself that any optimization problem written in either of these forms can be translated into one formulated in the inequality form (1) used in these notes, and hence that we can restrict our focus to only the form (1) without loss of generality.

1.5. Chapter End Notes

Linear programming has little to do with computer programming. Instead, the origin of the term 'program' lies with schedules and such. Think of an event programme, or think of television programming.

Although the LP concepts we discuss are universal, every textbook and lecture note uses different notation, different standards, and different words. This includes using the same words for slightly different concepts. This is an unfortunate situation, but there is little to be done about it at this point. We are carrying almost 80 year so historical baggage with us, and any attempt to fix it would be an instance of xkcd 927.

If you wish to refer to other sources besides these lecture notes, please **avoid Wikipedia on this topic**. Specifically avoid the pages on *linear programming*, *simplex method* and *revised simplex method*. They are poorly written and contain factual errors. I would hate for you to get confused by those.

2. The simplex method

The simplex method is a combinatorial algorithm which operates on the feasible set. We will develop the simplex method by considering first the relevant geometric concepts, and then talk through an example. The full algorithm is done afterwards. In the next lecture we prove that the simplex method terminates with an optimal solution.

2.1. Convex Polyhedra

The feasible set is a convex polyhedron. Convex: when you have two feasible points, then every point in the line segment connecting these two points is also feasible.

Polyhedron: you know that this is. Given a vector $a \in \mathbb{R}^d$ and a scalar $y \in \mathbb{R}$, the space $\{x \in \mathbb{R}^d : a^T x = y\}$ is called a hyperplane.[1] A set of the form $\{x \in \mathbb{R}^d : a^T x \leq y\}$ is called a halfspace. A set is a polyhedron if it can be written as the intersection of finitely many halfspaces.

Polyhedra have *faces*. If $P \subset \mathbb{R}^d$ is a polyhedron and $F \subset P$ then we say F is a face if, for every $x, x' \in P$ and every $\lambda \in (0,1)$, we have that $\lambda x + (1-\lambda)x' \in F$ implies $x, x' \in F$. Given an

[1] Based on your education before this, you may know this as an *affine* hyperplane and reserve the unqualified term for cases with $y = 0$. For our purposes that distinction makes little sense.

inequality description $P = \{x \in \mathbb{R}^d : Ax \leq b\}$, for every face $F \subset P$ there is an index set $S \subset [n]$ such that $F = \{x \in P : A_S x = b_S\}$. In the other direction, every subset of constraints will isolate a face.

Every face has a dimension. The polyhedron P is its own d-dimensional face. If you were imagining a P which is not d-dimensional, good job but in this course we will not bother with these edge cases. A $d-1$-dimensional face is called a facet. A 1-dimensional face is called an edge. A 0-dimensional face is called a vertex.

2.2. Development from Graphical Example

We assume we start with a *basis* of the LP. That is a set of indices $B \subset [n]$ of cardinality $|B| = d$ for which the square matrix A_B is invertible. From a basis we can extract the corresponding *basic solution* as $x^B = A_B^{-1} b_B$. We additionally assume that the basis is feasible. That is, we assume the basic solution satisfies $Ax^B \leq b$. These assumptions are not without loss of generality, but we will see later that they are also no big deal.

Finally, we assume that our current basic solution is not optimal. Otherwise we would already be finished. The simplex method works by changing its current basis into a better feasible basis, repeatedly until an optimal solution is reached.

Every vertex of the feasible set is a basic feasible solution: for any vertex $x' \in P$ there exists a basis $B \subset [n]$ with $x' = x^B$. Every facet of the feasible set corresponds with a linear constraint, a row from $Ax \leq b$.

By way of graphical example, let us take a cube. The starting basis consists of the indices for the constraints creating the bottom, front-left and front-right facets. The basic feasible solution is the vertex where these facets meet.

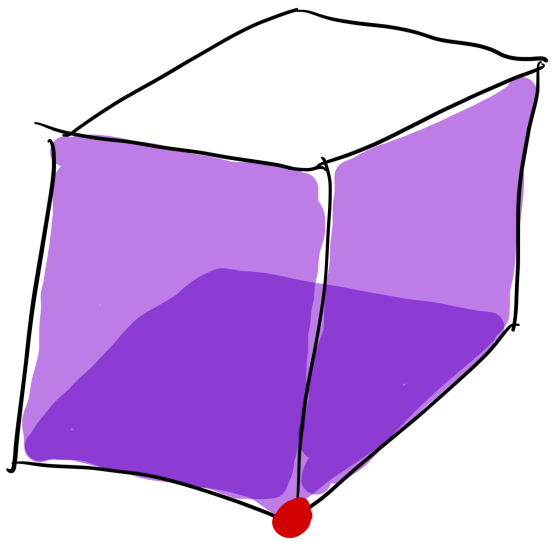


Figure 1: We start from the bottom vertex x^B, given by the basis B consisting of those constraints defining the bottom, front-left and front-right facets.

We wish to improve our basis and obtain a new vertex. Geometrically, we do this by leaving one facet, traversing along the edges where all remaining facets meet, and ending in the vertex on the opposite side of the edge. We change our basis by removing the index for the facet we left and adding the index for the facet we now touch. For the graphical example, let us move away from the bottom face and along the vertical edge. To accomplish this, let us first describe the direction in which we move.

We wish to find a vector $v \in \mathbb{R}^d$ such that the edge is part of the half-line $x^B + \lambda v, \lambda \geq 0$. Let $p \in B$ denote the index of the constraint creating the bottom facet. Then the vector $v \in \mathbb{R}^d$ must satisfy the following properties:

- $A_i v = 0$ for every $i \in B$ with $i \neq p$. This will ensure that $A_i(x^B + \lambda v) = A_i x^B = b_i$, so that the line segment stays inside the facets that it should stay inside.
- $A_p v = -1$ (or your favorite negative number of choice) will ensure $A_p(x^B + \lambda v) < b_p$, so that when we move in the direction of v then we leave the bottom facet p.

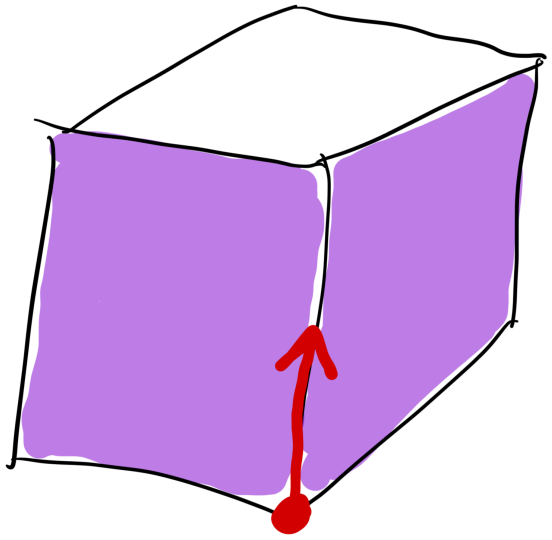


Figure 2: In order to take a pivot step we move away from one facet, the bottom facet. This constarint will leave the basis. The other two constraints remain in the basis. This defines an edge $x^B + \lambda v, \lambda \geq 0$, of the polyhedron that we will move across.

We can write the requirements for v more compactly as $A_B v = -e_p$, where $e_p \in R^B$ is the unit vector with a 1 on index p and zeroes at all other indices. One can solve this linear system as $v = -A_B^{-1} e_p$, meaning that v is the p-index column of the basis inverse matrix A_B^{-1}.

Now that we have determined the direction v, the next step is to compute the value of $\lambda \geq 0$ for which $x^B + \lambda v$ is the other endpoint of our chosen edge. For this, we must determine the largest value possible value λ for which the moved point is feasible $A(x^B + \lambda v) \leq b$. That is, we need $A_i(x^B + \lambda v) \leq b_i$ for every $i \in [n]$. Among these n inequalities, we need only consider those for which $A_i v > 0$. For those, we can rewrite the previous inequality to state

$$\lambda \leq \frac{b_i - A_i x^B}{A_i v} \quad \text{for all} \quad i \in [n] \quad \text{with} \quad A_i v > 0. \tag{4}$$

For the i with $A_i v \leq 0$, the corresponding inequality does not yield an upper bound on λ. Let λ^* denote the largest value for which all inequalities (4) hold.

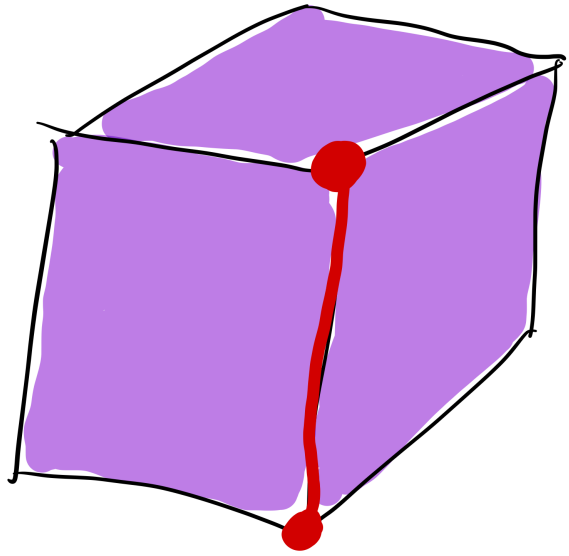


Figure 3: We move along the edge $x^B + \lambda v$ until hitting the other vertex of this edge. This is done by calculating the largest λ^* for which $x^B + \lambda^* v$ is in the feasible set. The constraint defining the top facet of the cube becomes tight, so this constraint will enter the basis.

Our next vertex of the feasible set is the point $x^B + \lambda^* v$ as calculated above. The constraint that will enter the basis, is the constraint $q \in [n] \setminus B$ for which (4) tight: $\lambda^* = \frac{b_q - A_q x^B}{A_q v}$.

To perform the simplex method, update $B \leftarrow B \setminus \{p\} \cup \{q\}$. Repeat until you are satisfied.

2.3. Full Algorithm

The simplex method keeps doing iterations[2] until no more improving moves are possible. That is, when there is no more possible choice of p for which $v = -A_B^{-1} e_p$ satisfies $c^T v > 0$, i.e., for which the next vertex found would have better objective value. That is, the method keeps going while there exists any i for which $\left(c^T A_B^{-1}\right)_i < 0$, and stops as soon as $c^T A_B^{-1} \geq 0$.

With that, we have all the pieces necessary to describe a simplex method.

SIMPLEX METHOD(A, b, c, B):

while $c^T A_B^{-1} \not\geq 0$:
  solve $A_B x^B = b_B$ for x^B
  choose $p \in B$ with $\left(c^T A_B^{-1}\right)_p < 0$ // This choice is called the 'pivot rule'
  solve $A_B v = -e_p$ for v
  set $\lambda^* = \min_{\substack{i \in [n] \setminus B \\ A_i v > 0}} \frac{b_i - A_i x^B}{A_i v}$ // This line and the next are the 'ratio test'
  choose $q \in [n] \setminus B$ with $\lambda^* = \frac{b_q - A_q x^B}{A_q v}$
  update $B \leftarrow B \setminus \{p\} \cup \{q\}$
return B.

The objective value at a basis B is $c^T x^B$, and the objective value at the next iteration will be $c^T(x^B + \lambda^* v)$. Hence, the increase in objective value is equal to $\lambda^* c^T v = -\lambda^* \cdot c^T A_B^{-1} e_p$. The term $-c^T A_B^{-1} e_p$ is for historical reasons called the *reduced cost*, and λ^* is called the *step size*. The objective increase is the product of the reduced cost and the step size.

[2] or linear algebraic reasons, these iterations are called "pivot steps"

2.3.1. But What If $\lambda^* = \infty$

This situation can happen when every $i \in [n] \setminus B$ satisfies $A_i v \leq 0$. In that case, the ray $x^B + \lambda v$ with $\lambda \in [0, \infty)$ certifies that the LP has no optimal solution. The algorithm may return 'unbounded'.

2.3.2. But What If $\lambda^* = 0$

If $\lambda^* = 0$ then that would mean we make a basis exchange $B' = B \setminus \{p\} \cup \{q\}$ without changing the current vertex $x^B = x^{B'}$. This is a situation that in theory is called *degeneracy*.

Degeneracy is a complex topic whose definitions, causes, implications and remedies are vastly different in theory and in practice. I am not aware of any comprehensive academic publications describing degeneracy in practice, and I do not respect the theoretical approaches to it. Thus, for this course, we will act as if **degeneracy does not exist**.

2.3.3. But What If q Is Not Unique

It theoretically could happen that there are multiple indices $q, q' \in [n] \setminus B$ with $\lambda^* = \frac{b_q - A_q x^B}{A_q v} = \frac{b_{q'} - A_{q'} x^B}{A_{q'} v}$. However, if that were to occur then in the next iteration we could have a situation with $\lambda^* = 0$. Thus, q not being unique qualifies as degeneracy. See the previous subsubsection why degeneracy does not exist.

In high-performance codes, choosing q correctly is important. For this purpose, one uses variations on *Harris' ratio test*. Experts broadly agree that Harris' ratio test is required for any high-quality code, but they disagree on *why* it is beneficial.

2.3.4. But How Do I Choose p

There is a freedom to choose any $p \in B$ which satisfies $\left(c^T A_B^{-1}\right)_p < 0$. The rule by which you choose p is called the *pivot rule*, and multiple pivot rules have been described over the decades.

- Dantzig, the originator of the simplex method, chose the $p \in B$ for which $\left(c^T A_B^{-1}\right)_p$ is minimal. Because he worked in the 'standard equality form' of LP, there were good reasons to call this the *most negative reduced cost pivot rule*.
- If alphabetical ordering is your favorite, you could consider choosing the smallest index $p \in B \subset [n]$ for which $\left(c^T A_B^{-1}\right)_p < 0$. This is called *Bland's least index rule* because Bland used this rule to study the simplex method under degeneracy (which, I remind you, does not exist).
- If you wish to maximize the amount of progress you make per pivot step, you could look ahead and choose the p for which the objective increase $-\lambda^* \cdot c^T A_B^{-1} e_p$ is maximized. This works to minimize the number of pivot steps but is very expensive computationally, since you must execute the ratio test once for every possible $p \in B$ in each iteration. This is called the *maximum increase pivot rule*.
- The geometrically cleanest choice is the *steepest edge pivot rule*. In this pivot rule we consider the normalized objective improvement per unit of movement $\frac{-c^T A_B^{-1} e_p}{\| -A_B^{-1} e_p \|}$.

In high-performance codes, you usually find variations on the steepest edge pivot rule. Assorted changes can be made to it in order to speed up its computation and make it more numerically stable.

2.3.5. But Is This Termination Criterion Sufficient?

This is an excellent question: just because none of the current vertex' outgoing edges gives an improving move (local optimality), is that sufficient to conclude that we have found an optimal solution (global optimality)? We will prove this in the next chapter.

2.3.6. But What If I Do Not Have A Feasible Basis To Start From

We can resolve this situation by writing a *two-phase simplex algorithm*. Pick your favorite basis $B^0 \subset [n]$, not necessarily feasible. Now write down the following *phase one* LP

$$\begin{aligned} &\text{minimize } s \\ &\text{subject to } Ax \leq b + s1_{[n]\setminus B^0} \\ &\qquad s \geq 0. \end{aligned} \tag{5}$$

where $1_{[n]\setminus B^0} \in \mathbb{R}^n$ indicates the vector all whose entries in B^0 have value 0 and all entries in $[n] \setminus B^0$ have value 1. Compute $s^{B^0} \in \mathbb{R}$ as the minimum value of s^{B^0} for which $Ax^{B^0} \leq b + s^B 1_{[n]\setminus B^0}$. Generically, there is a single index $i \in [n] \setminus B^0$ for which $A_i x^{B^0} = b_i + s^{\{B^0\}}$.

If $s^{B^0} \leq 0$ then B^0 was already feasible for (1), so the interesting case is to assume that $s^{B^0} > 0$. The index set $B^0 \cup \{i\}$ is a feasible basis for (5). (Convince yourself that this is a basis, and that it is feasible). You can now use the simplex method on this linear program (5) to find a feasible basis B for (5) with minimum possible value of s.

If that minimum value of (5) is 0, then among our optimal basis B of $d+1$ constraints we can find a subset of d indices which create a feasible basis for (1). (Convince yourself of this). From there, we have a feasible basis to start running the simplex method on (1).

If the minimum value is strictly greater than 0, then we have proven that (1) admits no feasible solutions. The optimal basis B of the phase one LP (5) will indicate a set of $d+1$ constraints in the phase two LP (1) which can not be satisfied simultaneously.

2.3.7. But What If There Are No Bases

This is possible when A does not have full column rank, but in my opinion this situation sounds contrived. If you do wish to handle this case, you can add additional constraints to the LP that constrain the lineality space.

2.3.8. But What Is The Running Time

In theory, each iteration can be completed in polynomial time. Hence when theorists talk about the running time of the simplex method, they are interested in the number of pivot steps required before the algorithm terminates. Decades of computational experience indicate that this number of pivot steps grows roughly linear with the problem size $d + n$. During this course, we will look at a number of theoretical investigations into this question.

In practice, the running time of the simplex method depends on your number of memory accesses. Unlike the vast majority of algorithms, the simplex method is *memory-bound*. Mostly your CPU idles while it waits for new data to arrive from RAM. This is partly why the ratio test is so expensive: it requires looking at the entire constraint matrix.

In real applications of LP, the constraint matrix is very sparse, with as little as 1% to 0.01% of entries being non-zero. Good implementations of the simplex method are aware of that sparsity and make good use of it.

3. LP Duality and Complementary Slackness

Consider an apple falling into a polyhedral fruit bowl. Under the influence of gravity, the apple finds its way to the lowest point it can reach. The apple has solved an optimization problem: it has maximized its position in the downwards direction, subject to being inside the bowl.

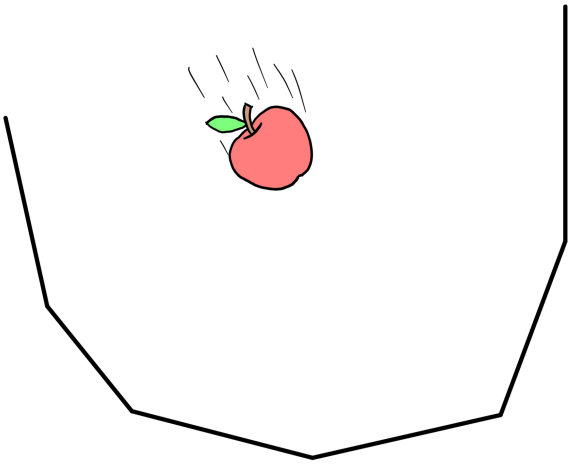


Figure 4: Throw the apple into the bowl.

When the apple has reached the lowest point, it will stay there at rest. The apple is no longer moving, which means that its acceleration is 0. From Newton's principle of $F = ma$, this implies that the net force acting on the apple is 0.

Two types of forces are acting on the apple. Gravity is pulling it downwards, and the normal forces from the bowl's walls are pushing the apple inwards into the bowl. Since the net force is 0, that means these forces exactly cancel each other out.

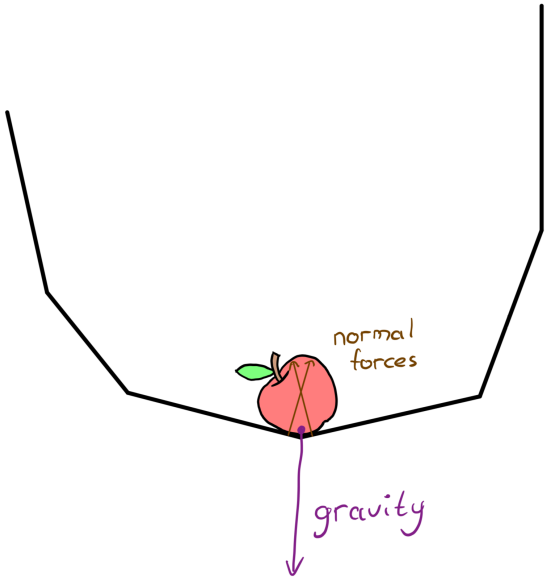


Figure 5: The apple is at rest. The normal forces (brown) cancel the force of gravity (purple). Note that these arrows are forces with lengths measured in Newtons while the rest of the picture (appple, bowl) have lengths measured in meters. Thus, the arrows indicate direction only; their length and position are somewhat arbitrary.

We write the apple-in-the-bowl problem as a linear program of the form (1). The objective vector c points down towards the ground. For every flat face of the fruit bowl, there is a corresponding inequality of the form $A_i x \leq b_i$. The apple lands in a location x^* which, subject to these inequality constraints, is the furthest possible in the downward direction. In other words, it maximizes $c^T x$ subject to $Ax \leq b$.

When the apple is at location x^*, it will touch some of the walls of the bowl. Let $S \subset [n]$ denote the indices of the corresponding constraints. That is, we have $S = \{i \in [n] : A_i x^* = b_i\}$.

Note that the constraint row vector A_i is an outward facing normal vector to the bowl wall. Because of that, we can write any inward-facing normal vector in our physics problem as $-y_i A_i$ for some $y_i \in \mathbb{R}$ with $y_i \geq 0$.

Newton's balance of forces, then, tells us the existence of a vector $y^* \geq 0$ such that

$$c^T + \sum_{i \in [n]} -y_i^* A_i = 0. \tag{6}$$

In more compact notation, this vector y^* satisfies the relation $A^T y^* = c$.

With all this in place, it is hopefully not a stretch to define the dual linear program. Its feasible set consists of all vectors that satisfy the specification we currently have of y.

Definition 1 (*Dual LP*)**.** *The original linear program (1) is called the* primal *LP. The corresponding* dual *LP is*

$$\begin{aligned} &\text{minimize } b^T y \\ &\text{subject to } A^T y = c \\ &\qquad\qquad y \geq 0 \end{aligned} \tag{7}$$

The dual linear program is an object of remarkable importance. It allows us to prove bounds on the optimal value of the primal LP (*the weak duality theorem*). We can find the optimal value of the primal LP by instead solving the dual LP (*the strong duality theorem*). Moreover, an optimal solution to one can tell us the structure of the optimal solutions to the other (*complementary slackness*).

Theorem 2 (*Weak duality*)**.** *Let $x \in \mathbb{R}^d$ be feasible for (1) and let $y \in \mathbb{R}^n$ be feasible for (7). Then $c^T x \leq b^T y$.*

Proof. Since $c = A^T y$ we can substitute $c^T x = y^T A x = \sum_{i \in [n]} y_i A_i x$. For every summand here, we know that $y_i \geq 0$ and $A_i x \leq b_i$. Hence $y_i A_i x \leq y_i b_i$ for every $i \in [n]$, and it follows that $\sum_{i \in [n]} y_i A_i x \leq y^T b$. □

Theorem 3 (*Strong duality*)**.** *Let $x^* \in \mathbb{R}^d$ be an optimal feasible solution for (1) and let $y^* \in \mathbb{R}^n$ be an optimal feasible solution for (7). Then $c^T x^* = b^T y^*$.*

Proof. The proof of strong duality requires the use of the following lemma, which we will not prove (but which does have an excellent Wikipedia page).

Lemma 4 (*Farkas' lemma*)**.** *Given a matrix $M \in \mathbb{R}^{k \times d}$ and a vector $c \in \mathbb{R}^d$, exactly one of the following is true:*

1. *There exists a vector $y \in \mathbb{R}^k$ satisfying $y^T M = c$ and $y \geq 0$.*
2. *There exists a vector $v \in \mathbb{R}^d$ satisfying $Mv \leq 0$ and $c^T v > 0$.*

Let $S = \{i \in [n] : A_i x^* = b_i\}$. To start, we prove that A_S and c cannot satisfy the second alternative (and hence must satisfy Farkas' first alternative).

If there existed a direction $v \in \mathbb{R}^d$ for which $A_S v \leq 0$ and $c^T v > 0$, then consider the point $x^* + \varepsilon v$ for ε to be determined. By assumption on v, we know that $A_S v \leq 0$ and hence $A_S(x^* + \varepsilon v) \leq b_S$. By construction of S we know that $\underset{S}{A_{[n]}} x^* < \underset{S}{b_{[n]}}$. Hence there exists some $\varepsilon > 0$ small enough for which $\underset{S}{A_{[n]}}(x^* + \varepsilon v) < \underset{S}{b_{[n]}}$. The point $x^* + \varepsilon v$ is thus a feasible solution to (1), but it has an objective value of $c^T(x^* + \varepsilon v) > c^T x^*$. This violates the assumption that x^* is optimal, and hence no such direction v can exist.

By Farkas' lemma, there must exist a vector $y_S \in \mathbb{R}^S$ with $y_S^T A_S = c$ and $y_S \geq 0$. We can amend this into a vector $y \in \mathbb{R}^n$ by setting $y_{[n] \atop S} = 0$. This vector satisfies $b^T y = b_S^T y_S = (A_S x^*)^T y_S = y_S^T A_S x^* = c^T x^*$. Observe that the vector y is feasible for (7). This proves that the optimal value of (7) is at least $b^T y^* \geq b^T y$. Weak duality tells us that $c^T x^* \geq b^T y^*$, and so we can go around and find $c^T x^* = b^T y \leq b^T y^* \leq c^T x^*$. This proves the theorem. □

Although we will not prove Farkas' lemma, please observe that it does exactly what Newton told us. The apple had no further direction v in which it could travel downwards ($c^T v > 0$) that wouldn't have it intersect the walls it was currently touching ($A_S(x^* + v) \leq b_S$, where S is the set of $i \in [n]$ with $A_i x^* = b_i$). In the physics story this meant the apple is at rest, hence the forces balance to zero. In Farkas' lemma, when we have an optimal point then the set of tight constraints there $A_S x \leq b_S$ allows for no further downward movement, hence the second alternative of Farkas is eliminated and the first alternative must be true. The equations $y^T M = c$ give us the same force balance as in the physics story.

Theorem 5 (*Complementary slackness*)**.** *Let $x^* \in \mathbb{R}^d$ be an optimal feasible solution for (1) and let $y^* \in \mathbb{R}^n$ be an optimal feasible solution for (7). Then for any $i \in [n]$, at least one of the following must hold: $y_i^* = 0$ or $A_i x = b_i$.*

Proof. Consider once again the equality $c^T x^* = \sum_{i \in [n]} y_i^* A_i x^*$. By strong duality, we know that $c^T x^* = b^T y^*$, and hence $\sum_{i \in [n]} y_i^* A_i x^* = \sum_{i \in [n]} y_i^* b_i$. For every individual $i \in [n]$, we already know that $A_i x^* \leq b_i$.

We prove complementary slackness by contradiction. Assume there is some $j \in [n]$ for which both $y_j^* > 0$ and $A_j x < b_j$. In that case we would have $y_j^* A_j x^* < y_j^* b_j$, which would imply that $\sum_{i \in [n]} y_i^* A_i x^* < \sum_{i \in [n]} y_i^* b_i$. We have found a contradiction, and hence no such j can exist. Therefore, for any $i \in [n]$ we have at least one of $y_i^* = 0$ or $A_i x = b_i$. □

Note that complementary slackness is clearly present already in the story of the fruit bowl. Only those parts of the fruit bowl that the apple touches ($A_i x^* = b_i$) can impart a normal force ($y_i A_i \neq 0$). In other words, when the apple does not touch a given facet of the bowl ($A_i x^* < b_i$) then that wall does not contribute to the normal force ($y_i A_i = 0$, hence $y_i = 0$). In our proofs, complementary slackness was a consequence of strong duality. Intuitively, I personally think of complementary slackness as preceding strong duality.

The dual LP is important for computational purposes. Somehow, for most applications, the simplex method on the dual LP is faster than it is on the primal LP. Because dual LPs tend to be structurally different from primal LPs, every high-quality software package for LP includes two simplex method implementations: a *primal simplex method* and a *dual simplex method.*

Mathematically the dual LP is also a rich object. In Alantha's lectures you will see some of its uses in approximation algorithms.

3.1. Certifying the optimal solution of an LP

The simplex method from last chapter terminates when it finds a feasible basis $B \subset [n]$ for which $c^T A_B^{-1} \geq 0$. When we define a vector $y^* \in \mathbb{R}^n$ by taking $y_B^* = A_B^{-T} c$ and $y_{[n] \setminus B}^* = 0$ then we can observe that y^* is dual feasible for (1). It satisfies complementary slackness, and hence it is an optimal dual solution with value $b^T y^* = c^T x^*$.

In other words, the simplex method terminates when its primal feasible basis gives a matching dual optimal feasible solution. This certificate allows the simplex method to prove that its primal solution is optimal too.

4. The Simplex Method is Slow

How many iterations does the simplex method require before it returns its solution? In practice, the answer is well-known to be roughly linear in the problem size (d, n), see [1] and the references therein. This observation was made already in the first years of the simplex method's use, and even now that LPs are 6 to 7 orders of magnitude larger than in the early days, nobody has seen fit to change the rule of thumb of a linear number of pivot steps.

Today we consider the simplex method under the model of *worst-case analysis*, and see that under this model the simplex method is inefficient. Note that everything we 'prove' today is false (it being mathematically true does not change that). The proofs are presented for mathematical curiosity only, and should not be taken as informative about the behavior of the simplex method.

4.1. Finiteness

First up: is the simplex method is guaranteed to terminate? Under the assumption of non-degeneracy, the answer is affirmative. Every basis $B \subset [n]$ has an associated solution x^B, and thus an associated objective value $c^T x^B$. After every iteration, the objective value of the current solution is strictly greater than in the previous iteration. It follows that no basis can ever be visited twice. Hence, the total number of iterations is at most the total number of bases, which is no greater than $\binom{n}{d}$.

This running time bound of $\binom{n}{d}$ can be improved slightly by counting only the number of feasible bases, instead of all possible bases. The maximum possible number of feasible bases is known exactly through McMullen's Upper Bound Theorem. I am not sure if there are any constructions tight with the UBT for which there is a path connecting all bases with increasing objective value. This may be an open question. But, if you don't care about matching the number exactly, then there are plenty of constructions of simplex paths of length $\left\lfloor \frac{2n}{d} \right\rfloor^{\lfloor \frac{d}{2} \rfloor}$ if the pivot rule makes the worst possible choice at every occasion.

What is a bit more interesting, is constructing LPs for which *a given pivot rule* will be forced into an exponentially long path.

4.2. Extended Formulation

We will prove that the steepest edge rule has exponential worst-case complexity, because steepest edge is the most interesting rule and I know a fun construction that we can use. The construction is based on an *extended formulation* by Ben-Tal and Nemirovski [2], [3], [4]. We will construct a polyhedron with $k + 2$ variables and $2k + 3$ inequalities, along with an orthogonal projection of that polyhedron to a 2-dimensional space. The projected polyhedron will be a regular polygon with 2^k vertices and edges.

To start, define for $i = 1, .., k$ the functionals $v_i = \begin{pmatrix} \sin\left(2^i \frac{\pi}{2^k}\right) \\ -\cos\left(2^i \frac{\pi}{2^k}\right) \end{pmatrix}$ and $w_i = \begin{pmatrix} \cos\left(2^i \frac{\pi}{2^k}\right) \\ \sin\left(2^i \frac{\pi}{2^k}\right) \end{pmatrix}$.

The polyhedron is defined inductively. For the base case, let $P_1 \subset \mathbb{R}^2$ be a triangle with constraints $P_1 = \left\{ (x, y) : x \geq 0, \begin{pmatrix} \cos\left(\frac{\pi}{2^k}\right) \\ \sin\left(\frac{\pi}{2^k}\right) \end{pmatrix}^T \begin{pmatrix} x \\ y \end{pmatrix} \leq 1, v_1^T \begin{pmatrix} x \\ y \end{pmatrix} \geq 0 \right\}$. This triangle describes angular section of the regular 2^k-sided polygon.

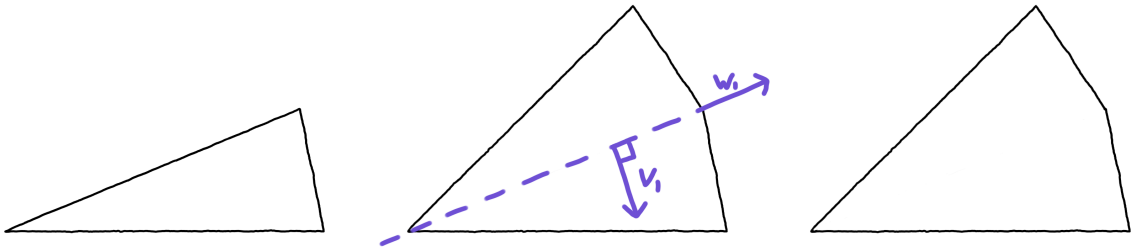


Figure 6: Left: P_1 for $k = 4$. Middle: we double the triangle. Right: P_2.

The induction case $P_{\{i+1\}}$ is obtained by effectively mirroring P_i along its counter-clockwise-most face. We let

$$P_{i+1} = \{p^{i+1} \in \mathbb{R}^2 : \exists p^i \in P_i \text{ such that } w_i^T p^i = w_i^T p^{i+1} \text{ and } v_i^T p^i \geq |v_i^T p^{i+1}|\} \tag{8}$$

That is, P_{i+1} consists of all those points $p^{i+1} \in \mathbb{R}^2$ for which there exists a mirror-image $p^i \in \mathbb{R}^2$ which is positioned at least as far below the line with normal v_i as p^{i+1} is away from that line, and which must lie in the induction case $p^i \in P_i$. You can verify that P_i is an angular slice of the 2^k-sided polygon with opening angle $2^i \frac{\pi}{2^k}$, and hence P_{i+1} is an angular slice of the 2^k-sided polygon with opening angle $2^{i+1} \frac{\pi}{2^k}$.

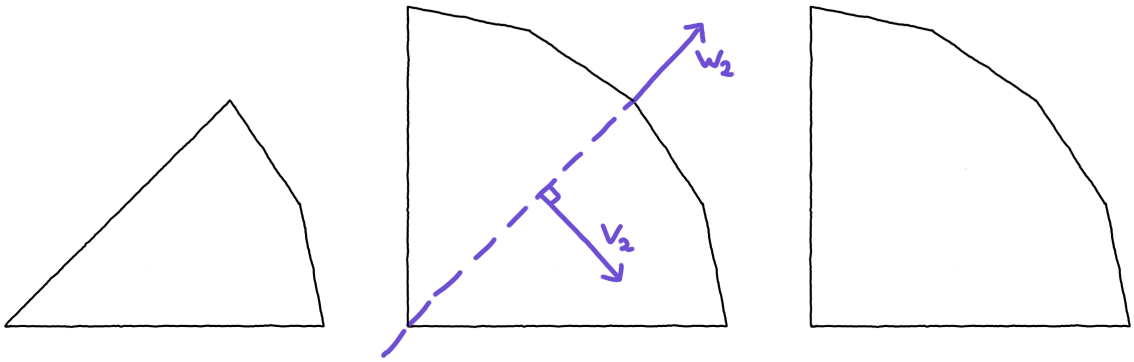


Figure 7: Left: P_2. Middle: we mirror for the induction step. Right: P_3.

Note that we can write this construction as a linear program. We introduce a pair of variables for the x and y coordinates of each $p^i \in P_i$ with $i = 1, ..., k+1$, counting $2k+2$ variables in total. All equalities of the form $w_i^T p^i = w_i^T p^{i+1}$ can easily be expressed as equality constraints in those $2k+2$ variables.

The inequality constraint $v_i^T p^i \geq |v_i^T p^{i+1}|$ use an absolute value operator, and so it is not linear constraints as desired for (1) yet. However, notice that the constraint is equivalent to the pair of constraints $v_i^T p^i \geq v_i^T p^{i+1}$ and $v_i^T p^i \geq -v_i^T p^{i+1}$, which are linear inequality constraints. This leads to the complete formulation

$$\left\{ (p_1, ..., p_{k+1}) \in R^{2k+2} : e_1^T p_1 \geq 0, \begin{pmatrix} \cos(\frac{\pi}{2^k}) \\ \sin(\frac{\pi}{2^k}) \end{pmatrix}^T p_1 \leq 1, v_1^T p_1 \geq 0 \right.$$
$$\forall i \in \{1, ..., k\} : w_i^T p^i = w_i^T p^{i+1}$$
$$v_i^T p^i \geq v_i^T p^{i+1} \tag{9}$$
$$\left. v_i^T p^i \geq -v_i^T p^{i+1} \right\}$$

using $2k + 2$ variables, k equality constraints, and $3 + 2k$ inequality constraints. These are all linear constraints suitable for a linear program (1).

The orthogonal projection onto the coordinates p_{k+1} is the desired one: the feasible set under this projection becomes a regular 2^k-sided polygon.

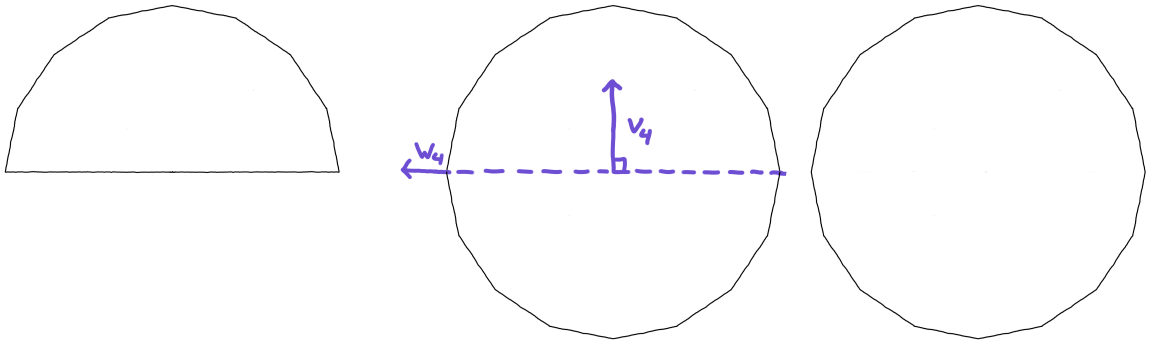


Figure 8: Left: P_4 is half the polygon. Middle: we double once more. Right: the final polygon P_5.

When we have a linear program using equality constraints, we can use those equalities to substitute out a number of variables. Since all k equality constraints are linearly independent, we can use that to reduce the system to a smaller one with $k + 2$ variables, $2k + 3$ inequality constraints, and no equality constraints.[3]

4.3. Running Time Lower Bound

Recall: the steepest edge pivot rule chooses as the leaving constraint, from among the indices $i \in B$ with $c^T A_B^{-1} e_i < 0$, the one for which the ratio $\frac{-c^T A_B^{-1} e_i}{\| -A_B^{-1} e_i \|}$ is maximal. The corresponding edge that the simplex method traverses, is the one with the lowest angle with the objective direction c.

Theorem 6 (*Steepest Edge Takes Exponential Time*)**.** *For every $d \in \mathbb{N}$ there exists a linear program of the form (1) in d variables with $O(d)$ inequality constraints such that the simplex method, with the steepest edge pivot rule, requires $\Omega(2^d)$ pivot steps when started at a particular basis.*

Proof. If we run the simplex method on a regular 2^k-gon, for arbitrary $k \geq 2$, in 2 variables, then the number of pivot steps in the worst case can easily be seen to be at least $\frac{2^k}{4} - 1$. Start the simplex method at the basis for the right-most basic feasible solution, and have the objective direction point upwards. The simplex method will trace the $\frac{2^k}{4}$ vertices from the right-most to the top-most vertex, and this takes $\frac{2^k}{4} - 1$ steps.

Now consider the feasible region described in (9). If we substitute the pivot rule's norm in $\frac{-c^T A_B^{-1} e_i}{\| -A_B^{-1} e_i \|}$ with the projected norm, measuring only the norm on the difference on the coordinates p_{k+1}, then

[3] Manually substituting out variables like this is appealing but only in theory. If you ever need to solve an LP in real life, you should probably trust your solver library to know what is best in terms of linear algebra.

you can easily verify that the same simplex pivot path described above remains valid. Start at the basis with the largest second coordinate in p_{k+1}, have the objective point towards having a high first coordinate in p_{k+1}, and the simplex method with steepness measure $\frac{-c^T A_B^{-1} e_i}{\|\pi_{p_{k+1}}(-A_B^{-1} e_i)\|}$ will take $\frac{2^k}{4} - 1$ pivot steps.

We wish to lower bound the number of pivot steps for the unscaled norm $\frac{-c^T A_B^{-1} e_i}{\| -A_B^{-1} e_i \|}$. To do this, we consider a scaling of the previous constraint set

$$\left\{ \begin{array}{r} (p_1, ..., p_k, \gamma \cdot p_{k+1}) \in R^{2k+2} : e_1^T p_1 \geq 0, \begin{pmatrix} \cos\left(\frac{\pi}{2^k}\right) \\ \sin\left(\frac{\pi}{2^k}\right) \end{pmatrix}^T p_1 \leq 1, v_1^T p_1 \geq 0 \\ \forall i \in \{1, ..., k\} : w_i^T p^i = w_i^T p^{i+1} \\ v_i^T p^i \geq v_i^T p^{i+1} \\ v_i^T p^i \geq -v_i^T p^{i+1} \end{array} \right\} \tag{10}$$

for some $\gamma > 0$ that we choose later. That is, everything stayed the same as in (9) except that the final two coordinates were scaled by a factor γ.

We can verify that some scaling by γ is sufficient to trick the steepest edge rule into following our desired (long) path. Instead of considering the ratio $\frac{-c^T A_B^{-1} e_i}{\|\pi_{p_{k+1}}(-A_B^{-1} e_i)\|}$, we can instead consider the ratio $\frac{c^{T(p-q)}}{\|} p - q\|$ for any two neighbouring vertices p, q of (10). You can see that any vertex of (10) projects down to the origin $\pi(p) = 0$ or to a vertex of the scaled regular 2^k-gon. Hence we find that $\|p - q\| \geq \|\pi(p - q)\| \geq \frac{2\gamma \sin\left(\frac{\pi}{2^k}\right)}{\cos\left(\frac{\pi}{2^k}\right)}$. For any edge that does not project onto the boundary of the projected 2^k-sided polygon, the length is at least $\sqrt{2}$ times as long.

At the same time, the entire feasible set of (9) is contained in $p \in [-2, 2]^{2k} \times [-2\gamma, 2\gamma]$. Thus, for any edge on our desired (long) path, the edge has Euclidean length at most $\frac{2\gamma \sin\left(\frac{\pi}{2^k}\right)}{\cos\left(\frac{\pi}{2^k}\right)} + 4\sqrt{2k}$. As soon as $\gamma \gg 1.5 \frac{4\sqrt{2k} \cos\left(\frac{\pi}{2^k}\right)}{2\gamma \sin\left(\frac{\pi}{2^k}\right)}$, the edges on our desired path are the steepest edges available. □

4.4. Bland's Rule

In the above, we proved that the steepest edge pivot rule has exponential worst-case complexity. Proving the same for Bland's rule is relatively easy. We can even prove that an entire class of LP problems, with uncertain non-zero entries, satisfies this.

Bland's pivot rule chooses as the leaving constraint, from among the indices $i \in B$ with $c^T A_B^{-1} e_i < 0$, the one for which numerical value of the index i is minimal.

Let's start with some traditional Klee-Minty cubes:

$$\begin{array}{l} \text{maximize } x_d \\ \text{subject to } 0 \leq x_1 \leq 1 \\ \qquad\qquad 0.2 x_{j-1} \leq x_j \leq 1 - 0.2 x_{j-1} \forall j \in \{2, ..., d\} \end{array} \tag{11}$$

Assume the inequalities are ordered as shown: first we list both inequalities for $j = 2$ and then both inequalities for $j = 3$ and so on.

We shift the means slightly and then perturb all non-zero coefficients. This yields a linear program

$$\begin{aligned} &\text{maximize } x_d \\ \text{subject to } 0.1+\alpha_1 &\le (1+\gamma_1)x_1 \\ (1+\delta_1)x_1 &\le 0.9+\varepsilon_1 \\ 0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} &\le \big(1+\gamma_j\big)x_j \\ \big(1+\delta_j\big)x_j &\le \big(0.9+\varepsilon_j\big)-\big(0.2+\eta_j\big)x_{j-1} \forall j \in \{2,\ldots,d\}. \end{aligned} \tag{12}$$

Theorem 7. *If $\alpha,\beta,\gamma,\delta,\varepsilon,\eta \in [-0.05,0.05]^d$ then Bland's rule takes exponentially many pivot steps.*

Proof. We go by induction. We prove that there are exactly 2^d bases, and Bland's rule will visit all bases when it starts at the basis minimizing x_d and works to maximize x_d. All basic feasible solutions x satisfy $x \in [0,1]^d$. The Bland path is reversible: if we start at the basis maximizing x_d and we use Bland's rule to minimize x_d then the same bases are visited but in reverse order.

We can check the base case $d=1$ easily. The starting basis consists of the constraint $0.1+\alpha_1 \le (1+\gamma_1)x_1$ and the end basis consists of the constraint $(1+\delta_1)x_1 \le 0.9+\varepsilon_1$. The two basic feasible solutions $\frac{0.1+\alpha_1}{1+\gamma_1}$ and $\frac{0.9+\varepsilon_1}{1+\delta_1}$ are easily verified to be in $[0,1]$.

When $dg=2$, assume that the induction statement has been proven for $d-1$. The starting basis for d consists of all lower-bounding constraints: $0.1+\alpha_1 \le (1+\gamma_1)x_1$ and $0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} \le \big(1+\gamma_j\big)x_j$ for all $j \in \{2,\ldots,d\}$.

Take the Bland path for $d-1$, and append to every basis in it the constraint $0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} \le \big(1+\gamma_j\big)x_j$. These bases are now bases for the KM-cube in d variables. The associated basic solutions are identical on coordinates $1,\ldots,d-1$ and have $x_d = \frac{0.1+\alpha_j+(0.2+\beta_j)x_{d-1}}{1+\gamma_j}$. The Bland path for $d-1$ contains $2^{\{d-1\}}$ bases and increases the value of x_{d-1} in every step. Since $1+\gamma_d \in [0.95,1.05]$ and $0.1+\alpha_d \in [0.05,0.15]$ and $0.2+\beta_d \in [0.15,0.25]$ and $x_{d-1} \in [0,1]$ we get $x_d \in [0,0.43]$.

When we consider the constraint $(1+\delta_d)x_d \le (0.9+\varepsilon_d)-(0.2+\eta_d)x_{d-1}$, we see that the left-hand side is at most $1.05 \cdot 0.43 \le 0.46$ and the right-hand side is at least $0.85-0.25=0.6$. That is all we needed to check in order to verify that the bases constructed so far are feasible.

Every step on our path so far is improving. Due to our chosen ordering of the inequalities, the constraint $0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} \le \big(1+\gamma_j\big)x_j$ will not leave the basis until there is no other possible improving direction left. Hence our path so far is all part of the Bland path for our KM-cube in d variables.

At the end of our path so far, Bland's rule will pivot out the constraint $0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} \le \big(1+\gamma_j\big)x_j$ and pivot in $(1+\delta_d)x_d \le (0.9+\varepsilon_d)-(0.2+\eta_d)x_{d-1}$, since it is the only available improving move.

Now take again the Bland path for $d-1$, but reverse its order and append to every basis in it the constraint $(1+\delta_d)x_d \le (0.9+\varepsilon_d)-(0.2+\eta_d)x_{d-1}$. This yields $2^{\{d-1\}}$ more bases for the d-variable KM-cube. Because we reversed the order, the Bland path is decreasing x_{d-1}. Since $0.2+\eta_d > 0$ and $1+\delta_d > 0$, the path is thus increasing the value of x_d. We have now constructed our full Bland path of 2^d bases. It remains to show that these final $2^{\{d-1\}}$ bases are feasible and in $[0,1]^d$. Note that $1+\delta_d \in [0.95,1.05]$, $0.9+\varepsilon_d \in [0.85,0.95]$ and $0.2+\eta_d \in [0.15,0.25]$. Since $x_{d-1} \in [0,1]$ we get $x_d = \frac{0.9+\varepsilon_d-(0.2+\eta_d)x_{d-1}}{1+\delta_d} \in [0.57,1]$. The only constraint that can lead to infeasiblity is $0.1+\alpha_j+\big(0.2+\beta_j\big)x_{j-1} \le \big(1+\gamma_j\big)x_j$ but its left-hand side is at most 0.4 and its right-hand side is at least 0.57, hence the bases are all feasible. □

4.5. Dantzig's Rule

Dantzig's pivot rule chooses as the leaving constraint, from among the indices $i \in B$ with $c^T A_B^{-1} e_i < 0$, the one for which the reduced cost $c^T A_B^{-1} e_i)$ is most negative.

Recall that Dantzig's pivot rule is not scaling-invariant. If we change a constraint from $A_i x \leq b_i$ to $\gamma A_i x \leq \gamma b_i$ for some number $\gamma > 0$, then the feasible region of the LP stays identical but the reduced cost $c^T A_B^{-1} e_i$ changes by a factor γ. This makes it conceptually simple to port over the behavior of Bland's rule on (11) to Dantzig's rule, by multiplying the various inequality constraints by large constants. In class we did not go over the details of how to do this.

4.6. Chapter End Notes

The finite termination argument we discussed at the start of this chapter is very pretty in theory. In practice it does not apply. The argument only works if, for every basis, we can associate a unique objective value. And moreover, if for every pivot step, we need to guarantee that the objective value of the new basis is strictly better than of the old basis. In practice, computing a basic solution like $x^B = A_B^{-1} b_B$ can only be done up to an accuracy of 10^{-6}. As such, if the objective value increase were less than this, then the actual objective value of the basis may get worse. Put in some additional practitioners magic, such as Harris' ratio test or bound perturbations, and any guarantee of finite termination is lost. The practitioners, of course, do not care. Their software works whether theory approves or not.

The extended formulation we described in this chapter was based on one by Ben-Tal and Nemirovski [2]. They introduced this object in order to approximate convex second-order programs like

$$\begin{aligned} &\text{maximize } c^T x \\ &\text{subject to } Ax \leq b \\ &\qquad\qquad \|Mx\|_2 \leq 1. \end{aligned} \tag{13}$$

The extended formulation allows to approximate the two-dimensional circular disk up to accuracy ε using $O\big(\log \frac{1}{\varepsilon}\big)$ variables and inequalities. Using standard tricks you can then approximate the d-dimensional sphere $\mathbb{B}_2^d$ up to accuracy ε with an extended formulation using $O\big(d \log \frac{1}{\varepsilon}\big)$ variables and constraints. This is asymptotically tight, which you can find by lower bounding the vertices necessary in the projection and upper bounding the maximum number of vertices (for a given number of variables and constraints) upstairs.

The Ben-Tal / Nemirovski [2] extended formulation, and the adapted version of Glineur [3], were introduced with the idea of solving convex optimization problems with second-order conic constraints $\|Mx\|_2 \leq r$ using only LP algorithms. These constructions used to be built-in features of the open-source MIP solver SCIP. That is, up to SCIP 7. In SCIP 8 they were removed because they didn't seem to bring much benefit.

To the best of my knowledge, the usage of these extended formulations for the purpose of simplex method lower bounds dates to [4]. Traditionally, simplex method lower bounds are proven using hypercubes with skewed faces. This dates back to work of Klee and Minty [5], and hence these objects are called Klee-Minty cubes. The Klee-Minty cube for the steepest edge rule was found by Goldfarb and Sit [6]. A broad overview of Klee-Minty cubes and a generalized construction can be found in this article of Amenta and Ziegler [7].

More difficult is proving lower bounds on the worst-case complexity of randomized pivot rules, or pivot rules which depend on the history of the trajectory thus far. These constructions are based on LP formulations for Markov Decision Processes. You can read more about those in, e.g., [8], [9].

5. The Simplex Method is Fast

Last lecture we saw an argument why the simplex method would be slow.

Definition 8. *Given a bounded linear program* $'\max' c^T x' s.t.' Ax \leq b$*, let* $\mathcal{B} = \mathcal{B}(A,b) \in \binom{[n]}{d}$ *denote the set of feasible bases. Define*

$$\gamma = \gamma(A,b) = \max_{i \in [n]} \max_{B \in \mathcal{B}} b_i - A_i x^B \tag{14}$$

to be the maximal slack among all feasible bases. Similarly, define the smallest non-zero slack at any basic feasible solution as

$$\delta = \delta(A,b) = \min_{i \in [n]} \min_{B \in \mathcal{B}} \min_{\substack{i \in [n] \setminus B \\ b_i \neq A_i x^B}} b_i - A_i x^B. \tag{15}$$

Recall that we are always assuming that degeneracy doesn't exist. In this case that means $\delta > 0$.

For the next proofs, lets consider the simplex method with Dantzig's pivot rule. Let us say that the algorithm is started from a feasible basis B^0, and that in every next iteration the pivot step is denoted by $B^{\{t+1\}} = B^t \cup \{q^t\} \setminus \{p^t\}$.

Let us abbreviate $x^t = x^{\{B^t\}}$, and write x^* to denote the optimal basic feasible solution on this simplex path (note, by finite termination the simplex path is finite and so x^* indeed exists).

Theorem 9. *The simplex method with Dantzig's pivot rule has its optimality gap shrink as* $c^T(x^* - x^t) \leq \left(1 - \frac{\delta}{d\gamma}\right)^t c^T(x^* - x^0)$.

Proof. If t is the final time step then the conclusion is immediate since $x^* = x^t$ and $c^{T(x^* - x^t)} = 0$. So assume that $t \geq 0$ is any time step before that moment. Let $N = \left\{i \in B^t : \left(c^T A_{B^t}^{-1}\right)_i < 0\right\}$ denote the indices of possible outgoing constraints. By assumption on t we know that $N \neq \varnothing$.

Consider the objective increase in one pivot step, which we can denote by $c^T(x^{t+1} - x^t) = c^T A_{B^t}^{-1} A_{B^t}(x^{t+1} - x^t)$. By construction of x^{t+1}, the vector $A_{B^t}(x^{t+1} - x^t)$ is zero on every coordinate except $p^t \in B^t$, where it is equal to $A_{p^t} x^{t+1} - b_{p^t} \leq -\delta$. Since p^t was selected using Dantzig's pivot rule, we know that it holds the most negative reduced cost and thus matches the infinity-norm $\left(c^T A_{B^t}^{-1}\right))_{p^t} = -\left\|\left(c^T A_{B^t}^{-1}\right))_N\right\|_\infty$.

Now bound the total objective gap as

$$\begin{aligned} c^T(x^* - x^t) &= c^T A_{B^t}^{-1} A_{B^t}(x^* - x^t) \\ &= \sum_{i \in B^t} \left(c^T A_{B^t}^{-1}\right)_i \cdot \left(A_{B^t}(x^* - x^t)\right)_i \\ &\leq \sum_{i \in N} \left(c^T A_{B^t}^{-1}\right)_i \cdot \left(A_{B^t}(x^* - x^t)\right)_i \\ &\leq \sum_{i \in N} \left\|\left(c^T A_{B^t}^{-1}\right)_N\right\|_\infty \cdot (b_i - A_i x^*) \\ &\leq d\gamma \left\|\left(c^T A_{B^t}^{-1}\right)_N\right\|_\infty. \end{aligned} \tag{16}$$

The first inequality uses that the entire vector $A_{B^t}(x^* - x^t)) = A_{B^t} x^* - b_{B^t} \leq 0$ is non-positive, meaning that the sum goes up when we drop terms outside N.

We now look at the fraction of the optimality gap closed in a single iteration. Using the above lower bound on the numerator and upper bound on the denominator we find

$$\frac{c^T(x^{t+1}-x^t)}{c^T(x^*-x^t)} \geq \frac{\delta \left\|\left(c^T A_{B^t}^{-1}\right)_N\right\|_\infty}{d\gamma \left\|\left(c^T A_{B^t}^{-1}\right)_N\right\|_\infty} = \frac{\delta}{d\gamma}. \tag{17}$$

Knowing that in every step a constant fraction of the gap is closed, we can now prove that the optimality gap shrinks exponentially. Using the above inequality we can state

$$\begin{aligned} c^T(x^*-x^{t+1}) &= c^T(x^*-x^t) - (c^T(x^{t+1}-x^t)) \\ &\leq c^T(x^*-x^t) - \frac{\delta}{d\gamma}c^T(x^*-x^t) \;\; = \left(1-\frac{\delta}{d\gamma}\right)c^T(x^*-x^t). \end{aligned} \tag{18}$$

This telescopes as $c^T(x^*-x^t) \leq \left(1-\frac{\delta}{d\gamma}\right)c^T(x^*-x^{t-1}) \leq \ldots \leq \left(1-\frac{\delta}{d\gamma}\right)^t c^T(x^*-x^0)$. □

Theorem 10. *Assume the rows of the constraint matrix A each have length at most $\|A_i\|_2 \leq 1$. The simplex method with Dantzig's pivot rule finds an optimal basic feasible solution after no more than $O\left(d^2\frac{\gamma}{\delta}\log\left(\frac{d\gamma}{\delta}\right)\right)$ pivot steps.*

Proof. Let B^* denote the final basis on the simplex path. By the simplex method's termination criterion we know that $c^T A_{B^*}^{-1} \geq 0$.

With this preparation out of the way, let us bound the amount of optimality gap progress that is possible before the algorithm needs to make 'combinatorial progress'. We can upper bound the initial optimality gap using Cauchy-Schwartz as

$$c^T(x^*-x^0) = c^T A_{B^*}^{-1} A_{B^*}(x^*-x^0) \leq \|c^T A_{B^*}^{-1}\|_1 \cdot \|A_{B^*}(x^*-x^0)\|_\infty \tag{19}$$

and hence $c^T(x^*-x^0) \leq d \left\|c^T A_{B^*}^{-1}\right\|_\infty \cdot \gamma$.

Consider the vector $z \in \mathbb{R}^{B^*}$ again. Let $p \in B^*$ be the index for which $\left(c^T A_{B^*}^{-1}\right)_p = \|c^T A_{B^*}^{-1}\|_\infty$. If $t \geq 0$ is any moment in time for which $B^t \not\ni p$, then we know

$$c^T(x^*-x^t) = c^T A_{B^*}^{-1} A_{B^*}(x^*-x^t) \geq \left(c^T A_{B^*}^{-1}\right)_p \cdot A_p(x^*-x^t) \geq \|c^T A_{B^*}^{-1}\|_\infty \cdot \delta \tag{20}$$

because the vector $z = c^T A_{B^*}^{-1} \geq 0$ is non-negative and $A_{B^*}(x^*-x^t) = b_{B^*} - A_{B^*}x^t \geq 0$ is too.

Putting the above inequalities together, we find that $p \notin B^t$ implies $\frac{c^T(x^*-x^t)}{c^T(x^*-x^0)} \geq \frac{\delta}{d\gamma}$. Using Theorem 9 we find that $p \notin B^t$ implies $t \leq \frac{\log\left(\frac{d\gamma}{\delta}\right)}{\log\left(1-\frac{\delta}{d\gamma}\right)} = O\left(d\frac{\gamma}{\delta}\log\left(d\frac{\gamma}{\delta}\right)\right)$.

Thus, after $O\left(d\frac{\gamma}{\delta}\log\left(d\frac{\gamma}{\delta}\right)\right)$ steps, the index p must forever onwards be contained in the basis. At that moment we can induct to a smaller LP with $d-1$ variables and $n-1$ inequalities. This induction case satisfies the same bounds of δ and γ on the slacks, and hence the simplex method must terminate at the optimal basis B^* in no more than $O\left(d^2\frac{\gamma}{\delta}\log\left(d\frac{\gamma}{\delta}\right)\right)$ pivot steps. □

My next lecture will be on September 29 or 30. Here, we will get started on modern theory and examine why the simplex method is fast. I have not yet decided for sure, but most likely we will start with the Dantzig / Ye / Kitahara and Mizuno upper bound on the running time for Dantzig's pivot rule.

Depending on the mood, afterwards we either talk about degeneracy and bound shifting, or we look at a probabilistic model. Or perhaps the Dadush and Hahnle analysis, that would be fun.

5.1. Chapter End Notes

The running time bound using γ and δ is from Kitahara and Mizuno [10]. It was originally developed in the language of equality-form LP by the same authors in [11], where they generalized a previous result by Ye [12] about the policy iteration method for optimizing Markov Decision

Processes. Ye's result for MDP's specifically was strengthened by [13] in a proof that I do not quite understand yet.

A very similar argument appeared previously in a retrospective paper of Dantzig [14], two decades before Ye. In this paper, he claims that the intuition for this argument goes back to 1947 when he originally invented the simplex method. It may be that this argument already appears in these old letters at the following URL, but I have not managed to verify this with certainty yet. https://sophie.huiberts.me/files/dantzig-von-neumann-1948.pdf

The computational results we discussed in class were by Kuno, Sano and Tsuruda [15]. Although the discussion was rather critical, I would not hold this against the authors. Computations such as these are difficult to do. It is better to have mediocre data than to have no data at all. And as far as I am aware, nobody else has even tried to do better.

6. Homework

My part of your homework is

1. Implement a simplex method, along with at least 2 different pivot rules
2. Run your simplex method on worst-cases instances for your pivot rules of choice. You may choose the worst possible starting basis. Plot the number of pivot steps against the number of variables in your LP problems.
3. Try to get the most pivot steps you can. The more pivot steps the better.
4. Write 1-4 pages about what you learned. Could you implement the simplex method as we saw on the board? Were any changes necessary to make it work at all? Did you make any changes that affected how many pivot steps you could perform?

Feel free to use any programming language and library that you want.

This homework can be performed without the use of chatboxes or robots. If you do use one of these tools, feel free to be a bit more ambitious with the homework. I trust you to make this work intellectually enriching for yourself.

If you get stuck, or you have any questions, then please reach out to me on sophie@huiberts.me

The full homework, including Alantha's part and the bonus part, can be found in your inboxes.

7. Shadow Vertex Pivot Rule

Today we cover the theoretically most studied pivot rule: the shadow vertex rule. We will discuss it from a perspective of invariants, geometrical interpretation, and progress measures. Compared to Dantzig's pivot rule running in $O\big(d^2 \frac{\gamma}{\delta} \log\big(d\frac{\gamma}{\delta}\big)\big)$ pivot steps, we will hopefully see that the shadow vertex rule can run in $O\Big(d\sqrt{\frac{\gamma}{\delta}\log\big(n\frac{\gamma}{\delta}\big)}\Big)$ pivot steps.

Recall that the simplex method always maintains the loop invariant of feasibility: at any moment the basis B satisfies $AA_B^{-1}b_B \leq b$.[4] For the shadow vertex method we have a second invariant involving a second objective vector $z \in \mathbb{R}^d$. We assume that, before the loop starts, the basis satisfies $z^T A_B^{-1} \geq 0$. Throughout the course of the loop, we assume there exists a number $t \in [0,1]$ such that $((1-t)z + tc)^T A_B^{-1} \geq 0$. Note that at the initial basis this is true with $t = 0$. Note also that, as soon as this invariant holds with $t = 1$ then the simplex method terminates since $c^T A_B^{-1} \geq 0$.

With this invariant in mind, we can design the shadow vertex pivot rule to make sure that invariant holds true. We find the maximum value of $t \in [0,1]$ for which the invariant holds, then we swap out

[4] There is much to say about this fact in floating-point arithmetic. Unsure if we will have time to discuss that.

the coordinate $p \in B$ on which the vector $((1-t)z + tc)^T A_B^{-1}$ reaches 0 (and hence would grow negative if t grew any further), and that is the entry we eliminate from the basis.

SHADOW VERTEX SIMPLEX METHOD$(A,\ b,\ c,\ z,\ B)$:

while $c^T A_B^{-1} \not\geq 0$:
  solve $A_B x^B = b_B$ for x^B
  set $t^{exit} = \max\{t \in [0,1] : ((1-t)z + tc)^T A_B^{-1} \geq 0\}$
  choose $p \in B$ with $\left(((1-t^{exit})z + t^{exit}c)^T A_B^{-1}\right)_p = 0$
  solve $A_B v = -e_p$ for v
  set $\lambda^* = \min_{\substack{i \in [n] \setminus B \\ A_i v > 0}} \frac{b_i - A_i x^B}{A_i v}$
  choose $q \in [n] \setminus B$ with $\lambda^* = \frac{b_q - A_q x^B}{A_q v}$
  update $B \leftarrow B \setminus \{p\} \cup \{q\}$
return B.

There are a few promises implicit in this design.
1. Such a t^{exit} and $p \in B$ exist.
2. Through this choice, the loop invariant is satisfied next iteration
3. The choice of p is unique.
4. This is a pivot rule: it satisfies $\left(c^T A_B^{-1}\right)_p < 0$.
5. There is a reason we call this the 'shadow vertex rule'.

Let's go through these one by one, and after that we cover the less obvious properties that this pivot rule has.

Lemma 11. *In the shadow vertex simplex method above, t^{exit} and $p \in B$ exist.*

Proof. By the loop invariant there is $t \in [0,1]$ with the property $((1-t)z + tc)^T A_B^{-1} \geq 0$. The set of all such t form a compact convex interval, so there is a maximum. If $t^{exit} = 1$ then that would imply $c^T A_B^{-1} \geq 0$ and we would not have taken another iteration of the loop, hence $t^{exit} < 1$. As such, there must be an inequality $p \in B$ that becomes tight $\left(((1-t)z + tc)^T A_B^{-1}\right)_p = 0$. □

Lemma 12. *The loop invariant is satisfied next iteration*

Proof. Now write $y = \left((1-t^{exit})z + t^{exit}c\right)^T A_B^{-1}$. We know that $y_p = 0$ and $(1-t^{exit})z + t^{exit}c = \sum_{\{i \in B\}} y_i A_i$. By construction we know that $y \in \mathbb{R}^B$ and $y \geq 0$. Let $B' = B \cup \{q\} \setminus \{p\}$ be the next basis visited and construct $y' \in \mathbb{R}^{B'}$ by $y_i = y'_i$ for all $i \in B' \cap B$ and $y'_q = 0$. Then we have $(1-t^{exit})z + t^{exit}c = \sum_{\{i \in B'\}} y_{i'} A_i$ immediately, indicating that $\left((1-t^{exit})z + t^{exit}c\right)^T A_{B'}^{-1} \geq 0$. Therefore the next basis also satisfies the loop invariant. □

For the third point, this is a matter of non-degeneracy. The only way for p not to be unique is if there were some silly numerical coincidence, i.e., a form of degeneracy, i.e., not interesting to worry about. This is a slightly different kind of degeneracy than what we talked about previously. So far, degeneracy has been a property of the pair A, b. The uniqueness of p is a property of the triple A, c, z.

Lemma 13. *The shadow vertex rule is a pivot rule*

Proof. For the fourth point, consider the affine function $t \mapsto \left(((1-t)z + tc)^T A_B^{-1}\right)_p$. If this function was constant 0 then there would be another $p' \in B$ that could be chosen, thus contradicting

uniqueness and non-degeneracy. We would not do such a thing. Therefore the affine function is non-constant, non-negative before t^{exit} and zero at t^{exit}. Since we have $\left(((1-t)z + tc)^T A_B^{-1}\right)_p = (1-t)(z^T A_B^{-1})_p + t(c^T A_B^{-1})_p$, the affine function on $t \in [0,1]$ is simply a convex combination of its values on its endpoints. The only way for this affine function to be strictly decreasing and zero at $t^{exit} \in [0,1]$ is if it is positive on its first endpoint $\left(z^T A_B^{-1}\right)_p > 0$ and negative on its second endpoint $\left(c^T A_B^{-1}\right)_p < 0$. This last property shows that p is a valid pivot direction. □

7.1. Why Is It Called That

The shadow vertex pivot rule was originally called, by Gass and Saaty, the *parametric objective rule.* Because there is an objective function and it is parametric. Not very inspired. But in 1977 Karl-Heinz Borgwardt wrote his PhD thesis on average-case analysis of the shadow vertex simplex method, and he dubbed it the 'schatteneckenalgorithmus', the shadow vertex algorithm.

The reason for that name is as follows. Take your feasible set $P = \{x \in \mathbb{R}^d : Ax \leq b\}$ and your objectives $c, z \in \mathbb{R}^d$. Let $\pi : \mathbb{R}^d \mapsto \text{span}(c, z)$ denote the orthogonal projection onto the two-dimensional linear subspace spanned by c and z.

We say that the polygon $\pi(P)$ is the 'shadow' of P. Here is how I drew this during lecture.

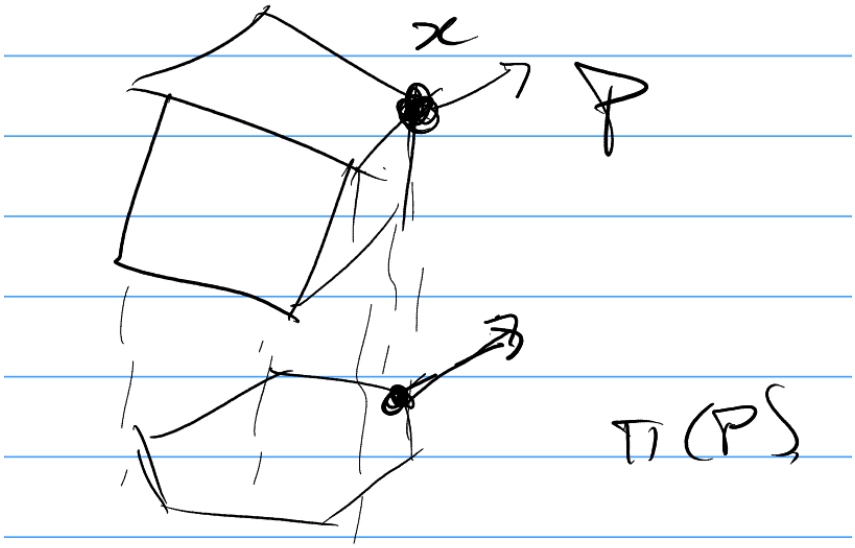


Figure 9: The feasible set P is a cube. It projects down to a hexagon. A point x is marked 'upstairs', and it projects to a vertex $\pi(x)$ downstairs.

Because $\pi(x)$ is a vertex, there is an objective vector $f \in \text{span}(c, z)$ who witnesses that $\pi(x)$ is a vertex of the projection $\pi(P)$. That same objective vector f witnesses that x is also a vertex of P.

Every vertex of the projection $\pi(P)$ maximizes some objective vector $f \in \text{span}(c, z)$. In particular, some of the vertices maximize objective vectors of the form $(1-t)z + tc$. These vertices of the projection $\pi(P)$ all lift to vertices of the feasible set P that maximize the same intermediate objectives.

All vertices visited by the shadow vertex simplex method have a representative on the shadow $\pi(P)$. That is why it is called the 'shadow vertex simplex method'.

One fun benefit of this shadow is this: if we want to bound the running time of the shadow vertex simplex method, it is sometimes easier to simply count the number of vertices of $\pi(P)$. Not for us here, but many papers have been written under this mindset.

7.2. Starting the Loop

As always with a loop invariant, we need to know why it should hold at the start of the loop. This boils down to choosing B and z.

If you already have a B and wish to run a shadow vertex method, you can pick your favorite non-negative vector $y \in \mathbb{R}^B_{\geq 0}$ and simply set $z = y^T A_B$. This works.

For our purposes, we will need to find a basis B that corresponds to a randomly sampled vector z. This is kind of just as difficult as optimizing the objective $c^T x$ in the first place. But not really: if you solve the phase one (Section 2.3.6) using the right shadow vertex simplex method as well, you can ensure that your initial feasible basis does what it needs to.

Phase one is always a bit of a chicken-and-egg situation, but you can always find your way out of the tangle. You can do everything if you believe in yourself.

7.3. Progress Measures

We saw in Lemma 13 that we pivot out elements $p \in B$ with $\left(c^T A_B^{-1}\right)_p < 0$ and $\left(z^T A_B^{-1}\right)_p > 0$. The former always implies that the objective value $c^T x^B$ is increasing over the course of the algorithm. The latter has a similar interpretation: the auxiliary value $z^T x^B$ is *decreasing* over the simplex method's running time.

This immediately yields two ways to measure progress of the shadow vertex simplex method: we make progress when $c^T x^B$ increases, and we make progress when $z^T x^B$ decreases. We can think of these as the 'loading bars' of the algorithm, they give an indication that progress is being made.

The shadow vertex method even admits a third progress measure, with the parameter t. We showed in the proof of Lemma 12 that t^{exit} of the previous basis is always part of the interval of t's for the current basis. That means that t^{exit} can only go up over the algorithm's lifetime, never down.

Usual simplex methods have only one monotone progress measure: the objective value $c^T x^B$. The fact that the shadow vertex simplex method has 3 is a great benefit for mathematical analysis of this algorithm.

7.4. Other Parametrization

For analytic simplicity, we will later use a different parametrization of the simplex method. You can see this variant below. It is essentially identical, except that previously we had $t \in [0, 1]$ and the intermediate objective was $(1-t)z + tc$ but now we take $t \geq 0$ and intermediate objective $z + tc$.

Different parametrizations can be useful in different occasions, but this $t \geq 0$ variant will be more useful for the remainder of our lectures. Please convince yourself that this algorithm is identical in every way that matters. The invariant is that there exists a $t \geq 0$ with $(z + tc)^T A_B^{-1} \geq 0$.

Shadow Vertex Simplex Method 2$(A,\ b,\ c,\ z,\ B)$:

while $c^T A_B^{-1} \not\geq 0$:
  solve $A_B x^B = b_B$ for x^B
  set $t^{exit} = \max\{t \geq 0 : (z + tc)^T A_B^{-1} \geq 0\}$
  choose $p \in B$ with $\left((z + t^{exit} c)^T A_B^{-1}\right)_p = 0$
  solve $A_B v = -e_p$ for v
  set $\lambda^* = \min_{\substack{i \in [n] \setminus B \\ A_i v > 0}} \frac{b_i - A_i x^B}{A_i v}$
  choose $q \in [n] \setminus B$ with $\lambda^* = \frac{b_q - A_q x^B}{A_q v}$
  update $B \leftarrow B \setminus \{p\} \cup \{q\}$
return B.

7.5. Worst-case Complexity

The extended formulation we saw previously can directly show that the shadow vertex rule has exponential worst-case complexity. Pick the starting and ending objective in the two-dimensional projection plane at a right angle, and the shadow vertex rule will visit a quarter of all vertices on the projection.

7.6. Random Vectors

In order to get a theoretically efficient simplex method, we need to randomize the shadow vertex rule. Essentially, the algorithm will choose the vector $z \in \mathbb{R}^d$ uniform from the sphere. You learned in Alantha's lectures that sampling a random unit vector is quite easily done.

Analytically, we will sample the vector $z \in \mathbb{R}^d$ from a Laplace distribution. The probability density function we choose to be proportional to

$$z \mapsto e^{-\|z\|_2}. \tag{21}$$

Let's do a quick calculation to show that this integral is finite, hence that there is a probability density function proportional to it.

$$\begin{aligned}
\int_{\mathbb{R}^d} e^{-\|z\|_2} \,\mathrm{d}z &= \int_0^\infty \int_{r\mathbb{S}^{d-1}} e^{-r} \,\mathrm{d}\theta \,\mathrm{d}r \\
&= \int_0^\infty r^{d-1} e^{-r} \int_{\mathbb{S}^{d-1}} \mathrm{d}\theta \,\mathrm{d}r \\
&= \int_0^\infty r^{d-1} e^{-r} \mathrm{vol}_{d-1}\left(\mathbb{S}^{d-1}\right) \mathrm{d}r \\
&= \mathrm{vol}_{d-1}\left(\mathbb{S}^{d-1}\right) \cdot (d-1)!.
\end{aligned} \tag{22}$$

That final equality is one that you know by looking it up on Wikipedia. The Gamma function is defined as that integral, and its value at integer values are the factorials.

(lmao I wrote this wrong the first time around, and its been broken for a good long time because I only fixed it in November. I put a $d!$ when it should've been a $(d-1)!$. Should have read wikipedia better.)

We see that the integral is finite, and hence there is a probability density function

$$z \mapsto \frac{1}{\text{vol}_{d-1}(\mathbb{S}^{d-1})(d-1)!} e^{-\|z\|_2}. \tag{23}$$

I don't know if it is easy to sample from this distribution. But again we don't actually care, the shadow vertex simplex method doesn't actually care about the norms $\|c\|, \|z\|$ of its objectives. So it is fine if the norms in the algorithm and the norms in the analysis are different.

7.7. Chapter End Notes

The shadow vertex pivot rule was first introduced as the 'parametric rule' by Gass and Saaty [16]. It was renamed as the shadow vertex (*schattenecke*) by Borgwardt in his PhD thesis [17]. Borgwardt used the shadow vertex rule to study the average-case running time for the simplex method, where he would assume that $b > 0$ and that the rows of A are drawn independently from a probability distribution spherically symmetric around the origin. Borgwardt went on to spend a substantial portion of his career on this topic and wrote an influential book about average-case complexity [18].

Borgwardt's finding that that the shadow vertex pivot rule is so amenable to probabilistic analysis has made this into the most well-studied pivot rule in theory [1], [19], [20].

Except for Erling Andersen, I think no practitioner likes the shadow vertex rule. It is hard and unpleasant to maintain the additional loop invariant.

The simplex method with the shadow vertex pivot rule has exponential worst-case running time. For an adversarial choice of objectives c, z, this can be shown using the exact same extended formulation we used for the steepest edge rule. Except that for the shadow vertex rule it is easier because there is no need to scale the variables to trick the norm.

In a very recent work, Black and Xue [21] showed that the adversarial choice of z is not necessary. They show that there are LPs A, b, c such that for any z, the shadow vertex path from z to c has exponential length.

8. The Simplex Method is Fast (2)

Last lecture we talked about the shadow vertex simplex method and its three progress measures. Today we use them to show that, if the auxiliary objective z is chosen at random, the shadow vertex rule takes roughly $O\big(d\sqrt{W/\delta}\log(\frac{1}{\varepsilon})\big)$ pivot steps in expectation. Here W is the mean width of the feasible set, which we define below. This is not quite what I promised last lecture lmao. Sometimes you promise something and it doesn't work out.

Like before, we assume the rows of A have norm $\|A_i\|_2 = 1$, the feasible set $P = \{x \in \mathbb{R}^d : Ax \leq b\}$ is bounded, and δ is the smallest non-zero slack among the feasible bases $\mathcal{B}$.

We use the second parametrization from last lecture, meaning that the intermediate objective is written as $z + tc$ for $t \geq 0$. For any basis B we write

$$y^B(t) = (z + tc)^T A_B^{-1} \in \mathbb{R}^B \quad \text{and} \quad I_B = \{t \geq 0 : y^B(t) \geq 0\}. \tag{24}$$

Every entry of $y^B(t)$ is affine in t, so I_B is a closed interval. When it is non-empty we write it as $I_B = [t_B^{enter}, t_B^{exit}]$.

Let $B^1, \ldots, B^k$ denote the bases visited by the algorithm, and abbreviate $x^i = x^{B^i}$. We know the following from last lecture. By the loop invariants, every B^i is feasible and has $I_{B^i} \neq \varnothing$. By Lemma 12, we essentially know that $I_{B^{i+1}} = \big[t_{B^i}^{exit}, t_{B^{i+1}}^{exit}\big]$, except for the first interval which starts at 0 and the last interval which is unbounded. These intervals partition the set of all $t \geq 0$ and so t^{exit} only goes up over the iterations.

The objective value $c^T x^i$ only goes up and the auxiliary value $z^T x^i$ only goes down. This is because of what we saw in Lemma 13, where p leaves the basis B then $\left(c^T A_B^{-1}\right)_p < 0 < \left(z^T A_B^{-1}\right)_p$.

Thus, we will have 3 different potential functions ('progress bars' or 'progress measures') for our algorithm. The objective value $c^T x^B$ goes up from start to finish, the auxiliary objective value $z^T x^B$ goes down, and t goes up.

8.1. A Random Objective

Little bug fix regarding last. We sample Z from the Laplace distribution, which has probability density function

$$f(z) = \frac{1}{\mathrm{vol}_{d-1}(\mathbb{S}^{d-1})(d-1)!} e^{-\|z\|_2}. \tag{25}$$

(note that we have $(d-1)!$ and not $d!$ like I wrote last time.) The probability density f only depends on $\|z\|_2$, which means that the direction $Z/\|Z\|_2$ is uniform on the sphere and independent of the length $\|Z\|_2$. The same calculation as last lecture, with one more factor r in the integrand, shows that $\mathbb{E}[\![|Z\|_2] = d$.

The shadow vertex simplex method only visits feasible bases B for which there exists a t with $(z+tc)^T A_B^{-1} \geq 0$. In the lecture I called this the "bad event", because every time it happens we have to take a pivot step. The following lemma says that a much stronger event, where every entry of $(z+tc)^T A_B^{-1}$ is at least some $\alpha > 0$, is almost as likely. This latter event I called the "good event" for reasons that will become clear soon.

Lemma 14 (*Shifting*)**.** *For every basis B, every $\alpha > 0$ and every set $T \subseteq [0,\infty)$ we have*

$$\Pr[\exists t \in T : (Z+tc)^T A_B^{-1} \geq 0] \leq e^{\alpha d} \Pr[\exists t \in T : (Z+tc)^T A_B^{-1} \geq \alpha \mathbf{1}]. \tag{26}$$

Note: the inequality in the other direction holds without the factor $e^{\alpha d}$. That is because $\alpha \mathbf{1} \geq 0$, so the latter event is included in the former.

Proof. Write Bad_B and Good_B for the sets of $z \in \mathbb{R}^d$ for which the first and the second event happen, respectively. Define the shift $h = \alpha A_B^T \mathbf{1} = \alpha \sum_{i \in B} A_i^T$. For any t we have

$$(z+h+tc)^T A_B^{-1} = (z+tc)^T A_B^{-1} + \alpha \mathbf{1}^T A_B A_B^{-1} = (z+tc)^T A_B^{-1} + \alpha \mathbf{1}^T, \tag{27}$$

so $z \in \mathrm{Bad}_B$ if and only if $z + h \in \mathrm{Good}_B$. Whenever you need to deal with continues probabilities, life is integrals. In particular, the probability of an event is the integral of that event's indicator function. Substituting $z = z' + h$ and using the triangle inequality $\|z' + h\|_2 \leq \|z'\|_2 + \|h\|_2$, we find

$$\begin{aligned}
\Pr[Z \in \mathrm{Good}_B] &= \int_{\mathbb{R}^d} f(z) \mathbb{1}[z \in \mathrm{Good}_B] \, \mathrm{d}z \\
&= \int_{\mathbb{R}^d} f(z' + h) \mathbb{1}[z' \in \mathrm{Bad}_B] \, \mathrm{d}z' \\
&\geq e^{-\|h\|_2} \int_{\mathbb{R}^d} f(z') \mathbb{1}[z' \in \mathrm{Bad}_B] \, \mathrm{d}z' \\
&= e^{-\|h\|_2} \Pr[Z \in \mathrm{Bad}_B].
\end{aligned} \tag{28}$$

By the triangle inequality once more, $\|h\|_2 \leq \alpha \sum_{i \in B} \|A_i\|_2 = \alpha d$. □

During the lecture we proved the above lemma for a fixed t, and I didn't readily know how to deal with the quantifier $\exists t$. I did say it should be easy, and thats true. The shift h does not depend on t so it is literally immediate.

Now fix some $\varepsilon > 0$. Let $N_\varepsilon(z) \subset \mathcal{B}$ denote the set of bases that the algorithm visits before t^{exit} exceeds $1/\varepsilon$, i.e., the number of i with $t_{B^i}^{enter} \leq 1/\varepsilon$. We call a basis $B \in \mathcal{B}$ *good* if there exists a $t^{mid} \in [0, 1/\varepsilon]$ with $y\big(z + t^{mid}c\big)^T A_B^{-1} \geq \frac{1}{d}\mathbf{1}$, and we call such t^{mid} a *good time* for B. Let $G_\varepsilon(z) \subseteq N_{\varepsilon(z)}$ denote the set of good bases.

Lemma 15. *The expected number of visited bases satisfies* $\mathbb{E}[N_\varepsilon(Z)] \leq e \cdot \mathbb{E}[\![G_\varepsilon(Z)]\!]$.

Proof. For any z, every basis in $N_\varepsilon(z)$ is feasible and has $(z + tc)^T A_B^{-1} \geq 0$ for some $t \in [0, 1/\varepsilon]$. That means we can count its size as

$$|N_\varepsilon(z)| = \sum_{B \in \mathcal{B}} \mathbb{1}\big[\exists t \in [0, 1/\varepsilon] : (z + tc)^T A_B^{-1} \geq 0\big]. \tag{29}$$

Now let Z be the random vector, and note that $\Pr\big[\exists t \in [0, 1/\varepsilon] : (z + tc)^T A_B^{-1} \geq 0\big] = \mathbb{E}\big[\mathbb{1}\big[\big[\exists t \in [0, 1/\varepsilon] : (z + tc)^T A_B^{-1} \geq 0\big]\big]\big]$. The set $\mathcal{B}$ of feasible bases does not depend on z, which means that the sum over $B \in \mathcal{B}$ does not worry us. We apply Lemma 14 with $\alpha = 1/d$ and $T = [0, 1/\varepsilon]$ to get

$$\mathbb{E}[N_\varepsilon(Z)] = \sum_{B \in \mathcal{B}} \Pr[Z \in \mathrm{Bad}_B] \leq e \sum_{B \in \mathcal{B}} \Pr[Z \in \mathrm{Good}_B] = e \cdot \mathbb{E}[\![G_\varepsilon(Z)]\!]. \tag{30}$$

□

This is strong structural information. Instead of counting all bases that the algorithm visits, it suffices to bound the number of good bases. We only needed Z to be random for the above lemma. From now on we fix z, and we bound $|G_\varepsilon(z)|$ for all z uniformly.

A word on stopping at $1/\varepsilon$. If we stop the algorithm as soon as $t^{exit} \geq 1/\varepsilon$, then the final basis is optimal for the objective $c + \varepsilon z$. When ε is small enough, this is fine. Real solvers have floating point numbers and error tolerances, and the ε disappears into the logarithm.

8.2. Counting Good Bases

Fix z and ε. For a vector $w \in \mathbb{R}^d$ we write $\mathrm{width}_w(P) = \max_{x,x' \in P} w^T(x - x')$ for the width of P in the direction w. We also define

$$\tau = \frac{\delta}{d \cdot \mathrm{width}_c(P)}. \tag{31}$$

On the board I only counted good times in $[\varepsilon, 1/\varepsilon]$, to stay away from $t = 0$. Lemma 16 shows that this is not necessary: below τ, no basis other than the starting basis can be good.

Lemma 16. *If $B \neq B^1$ is good, then every good time t^{mid} for B satisfies $t^{mid} \geq \tau$.*

Proof. As in the previous proof, $\big(z + t^{mid}c\big)^T(x^B - x^1) = y^B\big(t^{mid}\big)(b_B - A_B x^1)$. Since $B \neq B^1$ we have $x^1 \neq x^B$, so there is some $i \in B$ with $b_i - A_i x^1 > 0$. Because $i \notin B^1$, this slack is at least δ. Every entry of $y^B\big(t^{mid}\big)$ is at least $1/d$, so

$$\big(z + t^{mid}c\big)^T(x^B - x^1) \geq \frac{\delta}{d}. \tag{32}$$

The starting point x^1 maximizes $z^T x$ over P, so $z^T(x^B - x^1) \leq 0$. It follows that $t^{mid} \cdot c^T(x^B - x^1) \geq \delta/d$, and since $c^T(x^B - x^1) \leq \mathrm{width}_c(P)$ we conclude that $t^{mid} \geq \tau$. □

Theorem 17. *For every $\rho > 0$, the number of good bases is at most*

$$|G_\varepsilon(z)| \le 2 + d\rho \ln\left(\frac{1}{\tau\varepsilon}\right) + \frac{\mathrm{width}_z(P)}{\rho\delta}. \tag{33}$$

Proof. Let B^{last} denote the visited basis with $t^{enter} \le 1/\varepsilon < t^{exit}$.

Let $B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\}$ and let t^{mid} be a good time for B. When the algorithm visits B, by Lemma 16 the algorithm visits B and $t^{mid} \ge \tau$.

Since $B \ne B^{last}$, we know that $t_B^{exit} \le 1/\varepsilon$, so the algorithm pivots away from B. Let $p \in B$ denote the leaving index and B^+ the next basis. As always, that means B^+ is the unique feasible basis with $B^+ \supset B \setminus \{p\}$.

We saw in the proof of Lemma 13 that $\left(z^T A_B^{-1}\right)_p > 0$ and $\left(c^T A_B^{-1}\right)_p < 0$. The multiplier $\left((tc+z)^T A_B^{-1}\right)_p = t\left(c^T A_B^{-1}\right)_p + \left(z^T A_B^{-1}\right)_p$ is a decreasing affine function of t.

Pick a number $\rho > 0$, which we will choose later. One of two things is true: either $0 < \left(z^T A_B^{-1}\right)_p < \rho$ or $\left(z^T A_B^{-1}\right)_p \ge \rho$. In either case we will make progress on one of our progress measures.

Case 1: $\left(z^T A_B^{-1}\right)_p < \rho$. Here we win because t grows by a constant factor. By our choice of t^{mid} we know that $\left(\left(t^{mid}c + z\right)^T A_B^{-1}\right)_p \ge \frac{1}{d}$. This quickly gives us that

$$\left(\left(t^{mid}c + z - \frac{1}{d\rho}z\right)^T A_B^{-1}\right)_p \ge 0 \tag{34}$$

and hence also $\left(\left(\frac{t^{mid}}{1-\frac{1}{d\rho}}c + z\right)^T A_B^{-1}\right)_p \ge 0$.

By definition of p, we know that if $\left((tc+z)^T A_B^{-1}\right)_p \ge 0$ for some $t \ge t^{enter}$ then the whole vector satisfies $(tc+z)^T A_B^{-1} \ge 0$. We have found that $t^{exit} \ge \frac{t^{mid}}{1-\frac{1}{d\rho}} \ge \frac{t^{enter}}{1-\frac{1}{d\rho}}$. Rearranging, taking logarithms, and using $-\ln(1-x) \ge x$, get us

$$\ln t_B^{exit} - \ln t^{enter} \ge \frac{1}{d\rho}. \tag{35}$$

How many bases $B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\}$ can land in case 1? Well, we have our progress bar that will tell us. Any progress measure argument is a telescoping sum, and I suspect also the other way around. We get

$$\begin{aligned} \ln\frac{1}{\varepsilon} - \ln\tau &\ge \sum_{\{B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\}\}} \ln\left(t^{exit,B}\right) - \ln\left(t^{enter,B}\right) \\ &\ge \sum_{\substack{B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\} \\ \text{case 1 applies}}} \frac{1}{d\rho}. \end{aligned} \tag{36}$$

So the total number of bases in case 1 can be no greater than $d\rho \ln\left(\frac{1}{\tau\varepsilon}\right)$.

Case 2: $\left(z^T A_B^{-1}\right)_p \ge \rho$. Here we win because $z^T x$ goes down a lot. Since $B \setminus \{p\} \subseteq B^+$, the vector $b_B - A_B x^{B^+}$ is zero on every coordinate except p, and on coordinate p it is at least δ (by definition of δ). Therefore

$$z^T\left(x^B - x^{B^+}\right) = \left(z^T A_B^{-1}\right)\left(b_B - A_B x^{B^+}\right) = \left(z^T A_B^{-1}\right)_p\left(b_p - A_p x^{B^+}\right) \ge \rho\delta. \tag{37}$$

How many bases $B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\}$ can land in case 2? Again, any progress measure is a telescoping sum.

$$\begin{aligned} \text{width}_{z(P)} &\geq \sum_{\{B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\}\}} z^T\left(x^B - x^{B^+}\right) \\ &\geq \sum_{\substack{B \in G_\varepsilon(z) \setminus \{B^1, B^{last}\} \\ \text{case 2 applies}}} \rho\delta. \end{aligned} \tag{38}$$

As such, the total number of bases in case 2 can be no greater than $\frac{\text{width}_{z(P)}}{\rho\delta}$.

To count the total number of bases, we add together the number of bases in case 1 and in case 2, and add another 2 for B^1 and B^{last}. □

8.3. The Expected Running Time

Let $M = \mathbb{E}_\theta[\text{width}_\theta(P)]$, where θ is uniformly distributed on the sphere $\mathbb{S}^{d-1}$. This number is called the *mean width* of P.

Theorem 18. *The shadow vertex simplex method, with Z sampled from the Laplace distribution, satisfies*

$$\mathbb{E}[N_\varepsilon(Z)] \leq 2e + 2ed\sqrt{\frac{M \ln\left(\frac{d \cdot \text{width}_c(P)}{\varepsilon\delta}\right)}{\delta}}. \tag{39}$$

Proof. The function $w \mapsto \text{width}_w(P)$ is positively homogeneous, and the length and direction of Z are independent, so

$$\mathbb{E}[\text{width}_Z(P)] = \mathbb{E}[\|Z\|_2] \cdot W = dW. \tag{40}$$

Fix any $\rho > 0$. Taking expectations in Theorem 17 and applying Lemma 15, we find that

$$\mathbb{E}[N_\varepsilon(Z)] \leq e\left(2 + d\rho \ln\left(\frac{d \cdot \text{width}_c(P)}{\varepsilon\delta}\right) + \frac{dM}{\rho\delta}\right). \tag{41}$$

Now it is time to choose ρ. The right-hand side is minimized at $\rho = \sqrt{M / \left(\delta \ln\left(\frac{d \cdot \text{width}_c(P)}{\varepsilon\delta}\right)\right)}$, which gives the claimed bound. □

The mean width is at most the diameter of P (because the width in every direction is at most the diameter), so the bound also holds with the diameter in the place of W. That is the version we saw during the lecture.

8.4. Chapter End Notes

This chapter is a simplified version of the Phase II analysis in [1]. In that paper we do not assume a lower bound δ on the slacks. Instead, the right-hand side b is randomly perturbed. Such perturbations are a normal thing to do when implementing simplex method codes. When you do it right, the perturbations make it so that 90% of bases visited on the path will have the non-zero slacks be bounded away from zero. In theory this is why perturbations are good and useful.

We had a couple of typos and errors in the original lecture on the whiteboard. The lecture notes should be correct.

9. Course End Notes

During class I said that Tuesday would be the final lecture and Wednesday the exam. But Wednesday is a holiday, so that means Tuesday will be the exam.

Bibliography

[1] E. Bach, A. E. Black, S. Huiberts, and S. Kafer, "Beyond Smoothed Analysis: Analyzing the Simplex Method By-the-Book," in *Proceedings of the 58th Annual ACM Symposium on Theory of Computing*, in STOC '26. ACM, Jun. 2026, pp. 222–233. doi: 10.1145/3798129.3800742.

[2] A. Ben-Tal and A. Nemirovski, "On polyhedral approximations of the second-order cone," *Mathematics of Operations Research*, vol. 26, no. 2, pp. 193–205, 2001, doi: 10.1287/moor.26.2.193.10561.

[3] F. Glineur, "Computational Experiments With a Linear Approximation of Second-Order Cone Optimization," technical report, 2000. [Online]. Available: https://web.archive.org/web/20220505114903/https://perso.uclouvain.be/francois.glineur/oldwww/Papers/Image0001.ps.gz

[4] S. Huiberts, Y. T. Lee, and X. Zhang, "Upper and Lower Bounds on the Smoothed Complexity of the Simplex Method," in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC)*, ACM, Jun. 2023. doi: 10.1145/3564246.3585124.

[5] V. Klee and G. J. Minty, "How good is the simplex algorithm," *Inequalities : III : proceedings of the 3rd Symposium on inequalities*, pp. 159–175, 1972.

[6] D. Goldfarb and W. Y. Sit, "Worst case behavior of the steepest edge simplex method," *Discrete Applied Mathematics*, vol. 1, no. 4, pp. 277–285, 1979, doi: 10.1016/0166-218X(79)90004-0.

[7] N. Amenta and G. M. Ziegler, "Deformed Products and Maximal Shadows," *Contemporary Math.*, vol. 223, pp. 57–90, 1998.

[8] Y. Disser, O. Friedmann, and A. V. Hopp, "An exponential lower bound for Zadeh's pivot rule," *Mathematical Programming*, Jul. 2022, doi: 10.1007/s10107-022-01848-x.

[9] Y. Disser, G. Loho, M. Maat, and N. Mosis, "Lower Bounds for Ranking-Based Pivot Rules," Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, p. 31:1–31:19. doi: 10.4230/LIPICS.STACS.2026.31.

[10] T. Kitahara and S. Mizuno, "On the number of solutions generated by the dual simplex method," *Operations Research Letters*, vol. 40, no. 3, pp. 172–174, May 2012, doi: 10.1016/j.orl.2012.01.004.

[11] T. Kitahara and S. Mizuno, "A bound for the number of different basic solutions generated by the simplex method," *Mathematical Programming*, vol. 137, no. 1–2, pp. 579–586, Aug. 2011, doi: 10.1007/s10107-011-0482-y.

[12] Y. Ye, "The Simplex and Policy-Iteration Methods Are Strongly Polynomial for the Markov Decision Problem with a Fixed Discount Rate," *Mathematics of Operations Research*, vol. 36, no. 4, pp. 593–603, Nov. 2011, doi: 10.1287/moor.1110.0516.

[13] T. D. Hansen, P. B. Miltersen, and U. Zwick, "Strategy Iteration Is Strongly Polynomial for 2-Player Turn-Based Stochastic Games with a Constant Discount Factor," *Journal of the ACM*, vol. 60, no. 1, pp. 1–16, Feb. 2013, doi: 10.1145/2432622.2432623.

[14] G. B. Dantzig, "Origins of the simplex method." ACM, pp. 141–151, Jun. 1990. doi: 10.1145/87252.88081.

[15] T. Kuno, Y. Sano, and T. Tsuruda, “Computing Kitahara–Mizuno’s bound on the number of basic feasible solutions generated with the simplex algorithm,” *Optimization Letters*, vol. 12, no. 5, pp. 933–943, May 2018, doi: 10.1007/s11590-018-1276-4.

[16] S. Gass and T. Saaty, “The computational algorithm for the parametric objective function,” *Naval research logistics quarterly*, vol. 2, no. 1–2, pp. 39–45, 1955.

[17] K. H. Borgwardt, “ Untersuchungen zur Asymptotik der mittleren Schrittzahl von Simplexverfahren in der linearen Optimierung,” Doctoral dissertation, 1977.

[18] K. H. Borgwardt, *The Simplex Method: A Probabilistic Analysis*, vol. 1. Springer-Verlag, Berlin, 1987. doi: 10.1007/978-3-642-61578-8.

[19] E. Bach and S. Huiberts, “Optimal Smoothed Analysis of the Simplex Method,” in *2025 IEEE 66th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, Dec. 2025, pp. 1829–1856. doi: 10.1109/focs63196.2025.00096.

[20] D. Dadush and N. Hähnle, “On the shadow simplex method for curved polyhedra,” *Discrete Comput. Geom.*, vol. 56, no. 4, pp. 882–909, 2016, doi: 10.1007/s00454-016-9793-3.

[21] A. E. Black and L. Xue, “Any Proof of Polynomial Hirsch Must be Completely Incoherent.” [Online]. Available: https://arxiv.org/abs/2607.11628